Requests of a FEATHER Must Flock Together: Batch Size vs. Prefix Homogeneity in LLM Inference

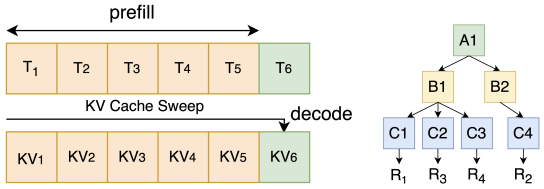
Anonymous Author(s)

Abstract

Auto-regressive token generation in large language models is memory-bound because it requires "attending to" key and value tensors (KV cache) of all previous tokens. Prior work aims to improve the efficiency of this *decode* process by batching multiple requests together, and maximizing batch size subject to GPU memory constraints. The key observation of our work is that with prefix-sharing workloads, smaller, prefix-homogeneous batches – where *all* requests share a common prefix – can achieve higher decode throughput than larger, heterogeneous batches, due to better spatial and temporal locality during KV cache accesses. However, prefix-aware schedulers in state-of-the-art inference engines maximize prefix reuse within a batch only to reduce KV cache memory footprint, but do not stop batch formation at smaller homogeneous batches that could have performed better. Further, we show that shared prefix detection in existing schedulers relies on radix-tree traversals, incurring substantial CPU overhead that is often comparable to GPU execution time. This paper presents FEATHER, a prefix-aware scheduler that uses reinforcement learning (RL) to learn the optimal tradeoff between batch size and prefix homogeneity. We also introduce Chunked Hash Tree (CHT), a lightweight data structure that enables fast prefix detection and efficient request selection for the RL scheduler, avoiding expensive tree traversals. We integrate FEATHER into vLLM and SGLang, and our evaluation shows that FEATHER achieves 2–10× higher end-to-end throughput as compared to existing schedulers, while doing no worse than the status quo when the workload does not have enough prefix sharing. FEATHER achieves these gains by reducing the total number of KV cache accesses, surpassing the performance of prefix-aware attention kernels that have the same goal.

1 Introduction

Large language models (LLMs) [12, 26, 36] are rapidly transforming modern computing, powering applications such as conversational assistants [10, 13], code generation [15], and retrieval-augmented generation [18, 39]. As these models transition from research to production, cloud AI engines [4, 14, 25] must improve serving latency and resource efficiency for both online and offline inference. While prior work has proposed scheduling strategies [1, 28, 45, 47], parallelization techniques [32, 37], memory management methods [17, 31, 42], quantization [9, 21], and kernel-level optimizations [27, 41], efficiently serving LLM workloads, especially under realistic high-throughput conditions, remains an open problem.



(a) Prefill and Decode **(b)** Radix Tree

Figure 1. LLM Inference Primitives

An LLM inference request goes through two phases: *prefill* and *decode*. The prefill phase processes all prompt tokens in parallel to produce the first output token, making it compute-bound. The decode phase, by contrast, generates tokens auto-regressively and is fundamentally memory-bound. Each newly generated token attends to the keys and values of all previous tokens, resulting in a sweep of the entire KV cache for each decode iteration, as shown in Figure 1(a). As context lengths grow, this memory traffic becomes the dominant bottleneck. Popular LLM inference engines like vLLM [17] adopt batching to improve GPU utilization during the decode phase, scheduling multiple requests together in a batch using policies like First Come First Serve (FCFS). More advanced frameworks explore techniques such as interleaving [1], disaggregation [47], and spatial co-scheduling [16, 22], along with dynamically adjusting batch sizes based on runtime constraints [28], to address this compute-memory mismatch.

Orthogonally, workloads such as few-shot prompting [8, 30], system prompts [33], and RAG [19] exhibit significant *prefix sharing* across requests. Reusing KV caches for shared prefixes eliminates redundant KV computation and reduces memory footprint. Systems such as SGLang [44] and vLLM [17] use radix trees (Figure 1(b)) or hashing-based structures to keep track of shared KV caches. While vLLM's FCFS would have scheduled requests in arrival order (R_1, R_2, R_3, R_4), SGLang performs a depth-first traversal of the radix tree and schedules requests in the order R_1, R_3, R_4, R_2, which reduces KV cache evictions. A complementary line of work [27, 38, 40, 41] exploits the shared prefix structure within the attention kernel itself by jointly computing attention for groups of queries that share prefixes. Such prefix-aware attention kernels avoid redundant loading of shared KV blocks for each request separately, thereby reducing memory traffic.

Despite these advances, we identify two critical gaps in current scheduling strategies for prefix-shared workloads. First, prior work largely focuses on maximizing batch size during the decode phase. However, we show via controlled experiments that prefix homogeneity — the extent to which *all* requests in a batch share a common prefix — has an

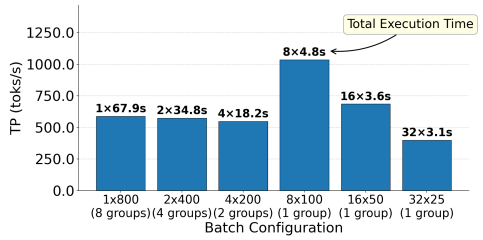


Figure 2. Batch Size vs. Prefix Homogeneity

even stronger impact on performance. Further, we find that the performance gains increase with the length of the prefix shared across all the requests in a batch (§3.1). This is because when a prefix is shared by all requests, there is significant *spatial* and *temporal* locality in the memory accesses of the KV cache located in the GPU DRAM, which leads to both higher memory bandwidth utilization and reduced total amount of data fetched from DRAM. As a result, moderately small prefix-homogeneous batches can leverage these locality benefits and perform better than larger heterogeneous batches. For example, Figure 2 shows the throughput and execution times of 800 requests, where 8 groups of 100 requests each share a large prefix of 10K tokens between them. Processing these requests in 8 prefix-homogeneous batches of 100 requests each leads to nearly 2× higher throughput than when using larger heterogeneous batches, though very small homogeneous batches (32 batches of 25 requests each) perform poorly as well due to sub-optimal compute utilization. Existing prefix-aware schedulers have no mechanism to recognize this tradeoff between batch size and prefix homogeneity (§3.2); e.g., in Figure 1(b), a DFS-traversal-based scheduler will use R_2 to maximize batch size, even if a smaller batch (only R_1, R_3, R_4) could have performed better due to a longer common prefix. The second big gap with current prefix-aware schedulers is that they incur prohibitive CPU overheads in maintaining and traversing radix trees, which can account for 50–90% of total latency (§3.3).

This paper introduces Feather[1] (§4), a prefix-aware scheduler that improves throughput by efficiently finding an optimal balance between batch size and prefix homogeneity. Feather is built on two components: *(i) Chunked Hash Tree (CHT)*, a lightweight data structure for fast prefix detection, similar in spirit to vLLM's block table [17]. Instead of performing token-level radix tree traversals, CHT represents each request as a vector of cumulative prefix hashes. To efficiently select a request that maximizes the length of the prefix shared across the active batch, CHT maintains a *min-heap*, updated lazily, and ordered by per-request *missing count*—the number of prefix chunks not yet covered by the batch. This enables the identification of the best candidate in $O(\log W)$ time among W waiting requests, a significant improvement over DFS traversal [44]. *(ii) RL-based batching policy*. Deciding when to stop adding requests to a batch presents a trade-off: larger batches improve utilization but dilute prefix homogeneity. We cast this as a reinforcement learning problem in which the scheduler observes batch characteristics (e.g., current size, shared prefix length), evaluates the impact of adding another request to the batch, and stops batch formation at a point that maximizes throughput. This adaptive approach consistently outperforms static heuristics across diverse workloads and hardware configurations. Together, these two components enable Feather to construct prefix-homogeneous batches efficiently, both improving system throughput and reducing scheduling overhead.

We implement Feather on vLLM [17] and SGLang [44] and evaluate it across diverse workloads and configurations (§5). Our results show that Feather achieves 2-10× higher end-to-end throughput over vLLM's FCFS and other baselines for shared prefix lengths ranging from 1K to 10K tokens, while also improving DRAM bandwidth utilization. Feather is fully hardware-agnostic and yet performs better than recent prefix-aware kernels that must be tuned to specific hardware configurations. Our RL policy ensures that our performance never falls below that of the baselines, even if the workload does not allow for prefix-homogeneous batches, and CHT reduces prefix-detection overhead by up to 1000× compared to DFS-based approaches for long sequences.

In summary, this paper makes the following contributions: (i) We demonstrate the tradeoff between batch size and prefix homogeneity in the decode phase performance of LLM inference, and identify previously overlooked CPU overheads in prefix-aware scheduling. (ii) We design the Chunked Hash Tree (CHT) data structure for low-overhead prefix detection, and develop a reinforcement learning-based policy for adaptive batch construction. (iii) We build Feather, a prefix-aware scheduler integrated into vLLM and SGLang, and demonstrate improvements in throughput and latency on diverse workloads. We plan to open-source Feather by contributing it to both frameworks' repositories.

2 Background and Related Work

We now discuss the background and prior work on batching, scheduling, and prefix-aware optimizations.

2.1 Background

LLM Inference and Attention. LLM inference proceeds auto-regressively in two phases: prefill and decode. Their latencies are known as *time to first token* (TTFT) and *time between tokens* (TBT), respectively. Each decode step comprises self-attention, Query-Key-Value-Output (QKVO) projections, and Multi-Layer Perceptron (MLP) layers. Self-attention captures global token dependencies as:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d}}\right)\mathbf{V}$$

where d denotes the head dimension. During the decode phase, the query $\mathbf{Q}$ of the current token attends to all prior keys $\mathbf{K}$ and values $\mathbf{V}$. These are cached in GPU memory as the

[1] Please see §E for an explanation of this name.

KV cache, eliminating redundant computation at the cost of increased memory usage. The asymmetry between prefill and decode, together with the growing memory footprint of the KV cache, motivates much of the prior work.

GPU Execution Model. GPUs are hierarchically organized for parallelism across hundreds of thousands of threads. The basic compute unit is the Streaming Multiprocessor (SM), and modern GPUs typically contain a few hundred SMs. Each SM includes an L1 cache, shared memory, tensor cores for matrix multiplication (GEMM), and execution units. SMs access global memory through a shared L2 cache. This hierarchy, from small, fast on-chip memory to large, high-latency global memory, is critical for LLM serving, as KV cache access patterns directly influence memory bandwidth utilization. Like CPUs, GPUs use hardware prefetching and benefit from regular, sequential memory access patterns [29].

2.2 Related Work

Batching and Scheduling. vLLM [17] adopts a first-come, first-served (FCFS) scheduling policy as a practical baseline for LLM serving. Sarathi-Serve [1] reduces prefill-decode interference through *chunked prefills* and stall-free batching, while DistServe [47] eliminates it by disaggregating the two phases across GPUs. At the kernel level, POD-Attention [16] overlaps prefill and decode on a single GPU via fused attention with SM-aware scheduling, and Bullet [22] enables fine-grained resource sharing through dynamic SM allocation. At the request level, BucketServe [45] groups requests by sequence length to improve batching efficiency, and Pang et al. [28] dynamically adapts the batch size to runtime constraints. Complementary to prior work, *FEATHER operates at the scheduler level and learns when to stop batch construction based on prefix homogeneity, introducing a scheduling signal not explicitly exploited in existing systems.*

Prefix Sharing. Many real-world LLM workloads [8, 19, 30, 33] exhibit substantial prefix overlap, and enabling KV cache reuse avoids redundant computation and reduces memory footprint. However, realizing these benefits requires coordination between serving and memory management. SGLang [44] formalizes prefix sharing via *RadixAttention*, which uses a radix tree to hierarchically organize token sequences, and enables KV reuse across shared prefixes. Similarly, vLLM [17] implements prefix caching [34] using a Merkle-tree-inspired hashing scheme (Figure 3), where each KV block hash depends on its tokens and preceding prefix hashes, enabling reuse when hash sequences match. BatchLLM [46] targets large offline workloads by grouping prefix-sharing requests and reordering them to maximize KV reuse. BlendServe [43] combines a resource-aware prefix tree with scheduling to jointly optimize prefix reuse and compute–memory demand. FEATHER adopts a lightweight design: *it uses chunked hashing over radix trees or full hash chains, reducing CPU overhead and steering batch construction toward prefix-homogeneous groups in online and offline settings.*

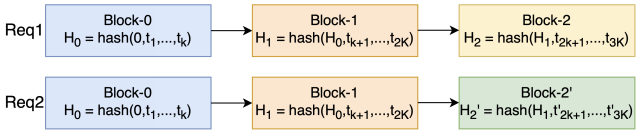


Figure 3. An Example of a vLLM Merkle Tree

Prefix-Aware Attention Kernels. Prior work [27, 38, 40, 41] also optimizes attention kernels by grouping queries with shared prefixes to load KV blocks once and reuse them across queries. These approaches, however, rely on GPU-specific kernel designs. In contrast, *FEATHER improves inputs to the existing attention kernels by constructing prefix-homogeneous batches upfront, making it hardware-agnostic and naturally composable with kernel-level techniques.*

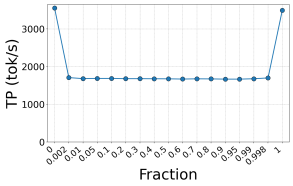
3 Motivation

Through a set of controlled experiments (here and in §B), we validate three hypotheses that underpin the design of FEATHER: (i) prefix homogeneity, the extent to which *all* requests in a batch share a common prefix, has a strong impact on the decode throughput because it increases the locality of KV cache accesses. Importantly, these gains can be achieved *without any modifications to existing attention kernels.* (ii) There is a trade-off between batch size and prefix homogeneity, where moderately sized homogeneous batches *can* outperform large heterogeneous batches; and (iii) existing prefix detection approaches incur substantial CPU overhead.

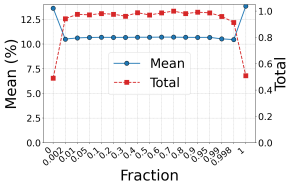
Experimental Setup. All experiments use the Llama 3 8B [2] model and are run on an RTX 6000 Ada GPU [23] with 48 GB of GDDR6 memory and a 96 MB L2 cache. We use the vLLM [17] inference engine with a default maximum batch size of 500 unless stated otherwise. We first focus on decode throughput, measured in output tokens per second. To isolate decode behavior, we perform a warm-up run so that the prefix KV caches are resident in GPU memory. This ensures that per-token compute remains constant within an experiment and differences arise primarily from memory accesses and batching. We also report end-to-end throughput, which includes the time taken to prefill the queries. For this model, a 10k-token request generates 1.2 GB of KV cache data across all layers. Within each layer, vLLM uses its paged memory management system and allocates the KV cache in blocks of 16 tokens contiguously in GPU memory, which is sufficient for locality benefits to kick in. We sample requests from the L-Eval [3] dataset and prune the size of each request as required by the experiment. We use the FlashAttention [11] backend throughout all the experiments.

3.1 Significance of Prefix Homogeneity

We start by observing the impact of prefix homogeneity on decode phase throughput. When *all* requests in a batch share a common prefix, their attention operations traverse the same KV cache regions concurrently. This yields two beneficial effects: (i) spatial locality: Sequential access to contiguous KV tensors enables the hardware prefetcher to



(a) Throughput



(b) Bandwidth Utilization

Figure 4. Prefix Homogeneity - Two Prefix Groups

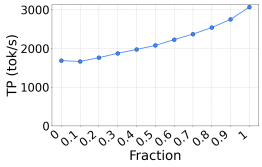


Figure 5. Fraction of Prefix Shared

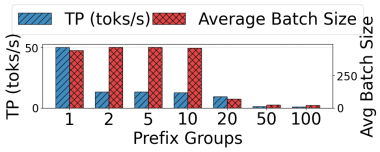


Figure 6. Number of Prefix Groups

fetch upcoming cache lines from DRAM into L2; (ii) temporal locality: Cache lines fetched for one request are reused by neighboring requests before eviction. Together, these effects increase L2 hit rates and DRAM utilization, improving throughput. In contrast, heterogeneous prefixes interleave KV accesses across disjoint regions, disrupting locality and reducing effective memory bandwidth. We will now present experiments that validate these claims.

Experiment 1: We consider two prefixes, A and B, each 10K tokens long. We form a batch of 500 requests, each using the prefix A or B, followed by a unique suffix of length 20 tokens (§A.1). Each request generates 50 decode tokens. Let $f \in [0, 1]$ denote the fraction of requests using prefix A. When f is 0 or 1, the batch is homogeneous; otherwise, two prefix groups coexist. We perform a warm-up run so that KV caches for both prefixes are resident in GPU memory.

Figure 4(a) shows decode throughput as a function of f. We see that the throughput is highest for fully homogeneous batches. However, even a single request from the other prefix group (e.g., $f = 0.002$) causes throughput to drop by nearly 2×. This demonstrates that decode throughput is highly sensitive to even minimal prefix heterogeneity. To understand this effect, we capture DRAM bandwidth utilization using DCGMI [24]. We first plot the mean bandwidth utilization for each f, and then we sum the utilization over the entire experiment duration (i.e., compute the area under the memory bandwidth curve) to approximate total memory accesses. As shown in Figure 4(b), homogeneous batches achieve over 40% higher bandwidth utilization than mixed-prefix batches due to greater spatial locality, enabling more efficient hardware-level prefetching into the GPU cache. Furthermore, greater temporal locality in prefix-homogeneous batches reduces the total data fetched from DRAM. Because most SMs process queries from the dominant prefix even when $f = 0.002$, we attribute the performance gains primarily to improved L2 cache reuse rather than increased L1 cache locality.

Takeaway 1: *Decode throughput is maximized when all requests share a prefix; even a single divergent prefix disrupts locality and sharply reduces effective bandwidth.*

Experiment 2: Next, we evaluate the impact of the shared prefix length on performance. For this, we construct a batch of 100 requests, each 2K tokens long and sharing a common prefix of length $2K \times f$ tokens, where $f \in [0, 1]$. The remaining $2K \times (1 - f)$ tokens are unique across requests (§A.2). The KV caches fit into GPU memory for all values of f. Figure 5 shows that throughput increases smoothly with f. It is lowest at $f = 0$ and improves steadily as sharing increases, reaching over 60% higher throughput at $f = 1$. As f grows, a larger portion of the KV cache is shared across requests; since decode repeatedly traverses the KV cache, greater spatial and temporal locality improve throughput.

Takeaway 2: *In prefix-homogeneous batches, decode throughput increases smoothly with the length of the shared prefix.*

Experiment 3: Next, we evaluate the effect of the number of prefix groups on performance using an experiment with N shared prefixes $P_1, P_2, \ldots, P_N$, each of length $10K$ tokens and having same number of requests. Figure 6 shows throughput and average batch size as N increases. As observed earlier, throughput drops from $N = 1$ to $N = 2$ due to reduced temporal and spatial locality arising from prefix heterogeneity. However, from $N = 2$ to $N = 20$, throughput remains stable, indicating that additional heterogeneity has a limited impact once locality is disrupted. Finally, throughput drops sharply from $N = 20$ to $N = 100$, as GPU memory can no longer hold KV caches for all active prefixes. The impact is also visible through the reduced average batch size for $N = 50$ and $N = 100$. Prior work [44, 46] mitigates the throughput drop from $N = 20$ to $N = 50$ due to KV cache evictions by maximizing prefix reuse and minimizing memory footprint using depth-first traversal of radix trees. However, to the best of our knowledge, no prior work has observed the drop in performance from $N = 1$ to $N = 2$ prefix groups.

Takeaway 3: *While prefix heterogeneity causes a drop in throughput, the amount of heterogeneity has a marginal impact on the loss of locality. However, very high heterogeneity leads to minimal KV reuse and higher GPU memory evictions.*

Experiment 4: Until now, all the experiments have considered requests that have a single level of prefix sharing. We now ask the question: what is the impact of more complicated sharing patterns on performance? For this, we construct radix trees with 1 to 4 levels of shared prefixes (§A.3). There are two branches in each level, and requests are equally distributed across the leaves. The sequence length per request is fixed at $4L$, and Figure 7 shows the decode throughput for three values of L. As expected, a single level of radix tree has the highest performance because of maximum locality. Interestingly, radix trees with 2, 3, and 4 levels exhibit similar throughput across different values of L, despite differing branching structures. Why so? During the initial segment of length L, all configurations access the same KV region. However, once branching occurs, the batch becomes heterogeneous, locality degrades, and the marginal cost of additional prefix groups is small, as established earlier in Figure 6.

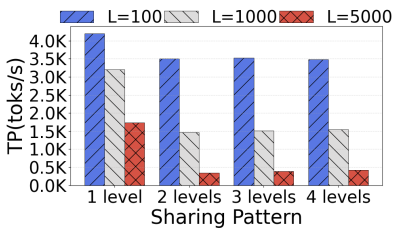


Figure 7. Radix Tree Sharing Patterns

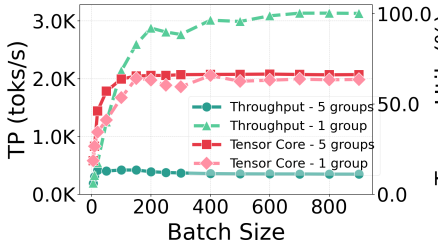


Figure 8. Throughput vs. Batch Size

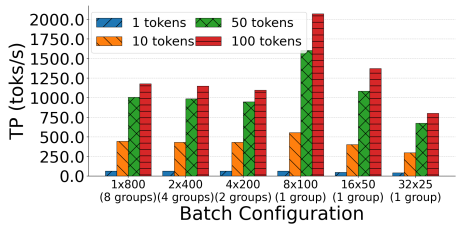


Figure 9. Batch Size vs. Homogeneity

Takeaway 4: *Performance is driven mainly by the length of the top-level prefix shared by all requests, while the number of radix-tree levels and branches has a negligible impact.*

3.2 Batch Size vs Prefix Homogeneity

So far, we have established that prefix homogeneity has a significant impact on performance. But when the workload does not have enough requests from a prefix group and prefix-homogeneous batches are too small, the compute will remain under-utilized, leading to a loss in throughput. Therefore, there exists a trade-off between batch size and prefix homogeneity. We now explore this trade-off.

Experiment 5: We study decode throughput vs. batch size using two workloads: (i) heterogeneous, with requests spread across 5 distinct 10K-token prefixes, and (ii) homogeneous, with all requests sharing one such prefix. Figure 8 shows throughput and tensor core utilization as a function of batch size for both workloads. We see that throughput plateaus at a point before the maximum batch size, and the saturation throughput is higher in the homogeneous case due to greater locality. Further, the batch size at which throughput saturates is higher for the homogeneous workload because requests can access the same KV cache without using additional memory bandwidth. Interestingly, the compute utilization is almost exactly the same across both workloads.

Takeaway 5: *Decode is memory bandwidth bound and increasing batch size beyond a point does not necessarily improve throughput. However, the point at which diminishing returns begin depends on prefix homogeneity (and hardware config).*

Experiment 6: Given that batch size has a diminishing impact on performance, we ask if moderately small homogeneous batches can outperform larger heterogeneous ones. To answer this, we use eight prefix groups with 100 requests each (800 total, each 10K tokens) and evaluate the following batching strategies: (i) 1×800 (8 prefix groups/batch); (ii) 2×400 (4 prefix groups/batch); (iii) 4×200 (2 prefix groups/batch); (iv) 8×100 (homogeneous); (v) 16×50 (homogeneous); and (vi) 32×25 (homogeneous). We vary the decode length from 1 to 100 tokens and measure the end-to-end throughput, including both the prefill and decode phases for all batches. Figure 9 shows this throughput across the different batching strategies. For small decode lengths, throughput is dominated by prefill, and large heterogeneous batches perform the best due to higher tensor core utilization; splitting 1×800 into smaller batches slightly reduces throughput due to lower compute efficiency, as locality benefits are not yet dominant. As decode length increases, performance trends reverse: multiple decode iterations repeatedly sweep the KV cache and reap greater benefits due to locality. Therefore, at longer decode lengths, fully homogeneous batches (8×100) significantly improve throughput despite smaller batch sizes. Notably, even 16×50 continues to outperform 1×800 at large decode lengths, showing that locality gains can outweigh reduced tensor core utilization. Only when batches become too small (32×25) does throughput decline again, as reduced arithmetic intensity and occupancy dominate.

Policy	Exec (s)	Batch Time (s, %)	Throughput (toks/s)
vLLM FCFS	229.36	3e-6 (0.00%)	436.25
SGLANG DFS-W	177.78	92.15 (51.8%)	562.61
SGLANG LPM	149.27	46.62 (31.2%)	669.92
PAT (vLLM Block Table)	316.45	91.72 (29.0%)	316.63
vLLM RadixTree	139.33	62.95 (45.2%)	717.35
vLLM User IDs	105.43	0.03 (0.03%)	948.54

Table 1. Overhead of Dynamic Prefix Detection

Takeaway 6: *Moderately small prefix-homogeneous batches can achieve higher throughput than larger heterogeneous ones. Gains increase with longer decode lengths as repeated KV sweeps amplify locality; however, excessively small batches underutilize compute and degrade performance.*

3.3 Overhead of Dynamic Prefix Detection

Next, we study how state-of-the-art inference engines identify prefix sharing and the costs involved. Systems such as SGLang [44] and BatchLLM [46] do not explicitly form prefix-homogeneous batches, but use radix trees to detect sharing, and schedule requests to minimize KV recomputation.

Experiment 7: We construct a workload of 2K requests, each with a 10K-token prefix, and compare scheduling CPU time to total execution time. SGLang proposes two prefix-aware policies that use its radix tree: *Longest Prefix Match*, which prioritizes requests with the longest matching cached prefixes, and *DFS-Weight*, which performs a depth-first traversal and prioritizes branches with more active requests. vLLM uses a much simpler hash-based block table, which is used for grouping requests in the prefix-aware attention kernel implementation of PAT [41]. We compare the overheads of this block table along with our own radix-tree-based scheduler implemented on top of vLLM. We also compare against an idealized policy where prefix group identifiers are provided explicitly, eliminating dynamic prefix discovery.

Table 1 shows that SGLang's LPM and DFS-Weight incur scheduler overhead (measured as time taken by all functions

invoked during batch formation) comparable to GPU execution time, even though these policies improve performance over vLLM's FCFS. In contrast, when prefix group identifiers are provided explicitly, CPU overhead drops significantly. This gap arises from the structure of radix trees: they require token-by-token comparisons during insertion, dynamic node splitting on partial matches, and repeated tree traversals for scheduling requests. With long prompts and high concurrency, these operations incur significant overhead. Similarly, vLLM's block table design is inefficient in this regard. However, these overheads are not sufficiently highlighted in prior work [43, 46], which is mostly based on offline inference.

Takeaway 7: *Existing approaches introduce significant CPU overhead when detecting prefix sharing, which is sometimes even comparable to the GPU execution time.*

4 Design and Implementation

Our experiments so far have demonstrated that the length of the prefix shared across *all* active requests is an important determinant of system performance. However, this metric interacts with batch size in a complex way: larger batches improve GPU utilization but often reduce shared prefix alignment. Further, detecting this shared prefix length incurs high overhead in existing systems. We now describe the design of Feather, an efficient scheduler that finds the right balance between batch size and prefix homogeneity.

4.1 Overview

Feather is based on the following two key ideas.

Key Idea 1: Learn the Stopping Decision via RL (§4.3). The trade-off between batch size and prefix homogeneity raises a key question: when should the scheduler stop adding requests? The answer depends on both hardware and workload, making fixed heuristics brittle. We model batch construction as a sequential decision process and learn a stopping policy via Reinforcement Learning. At each step, the scheduler evaluates a candidate request using various state features and decides whether it should be added to the batch or if the batch should be dispatched to the GPU for execution.

Key Idea 2: Shared Prefix Detection via Chunked Hash Tree (§4.2). Tree-based approaches incur significant overhead (§3.3), and finding the best waiting request requires a DFS traversal that is $O(E)$, where E is the number of edges in the radix tree. In the worst case, this can go up to $O(W \times T)$, where W is the number of waiting requests, and T is the sequence length. We instead represent each request as a sequence of hashes over fixed-size chunks and maintain a working set of hashes for the active batch. This is similar in spirit to Merkle-style hash chains, but it does not require sequential hash computation, and each chunk can be processed independently without prefix dependencies. Prefix matching is thus reduced to a set-overlap problem: selecting the request with the maximum overlap with the working set. However, a naive implementation that scans all requests and counts overlaps requires $O(W \cdot C)$ work per decision, where C is the number of chunks in a request, which remains too expensive for frequent scheduling. We avoid this by maintaining a missing count per request—the number of chunks absent from the working set—which changes only when the working set changes (i.e., upon request addition or completion). Overlaps are thus updated incrementally, reducing selection to finding the request with the smallest missing count. To do so efficiently, we maintain a min-heap over missing counts, pushing updates and using lazy deletion to discard stale entries, yielding $O(\log W)$ selection cost.

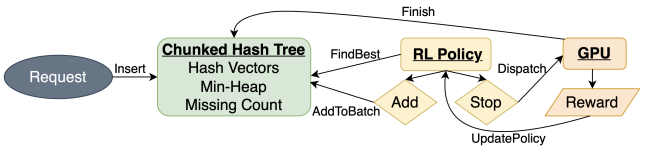


Figure 10. Feather Pipeline

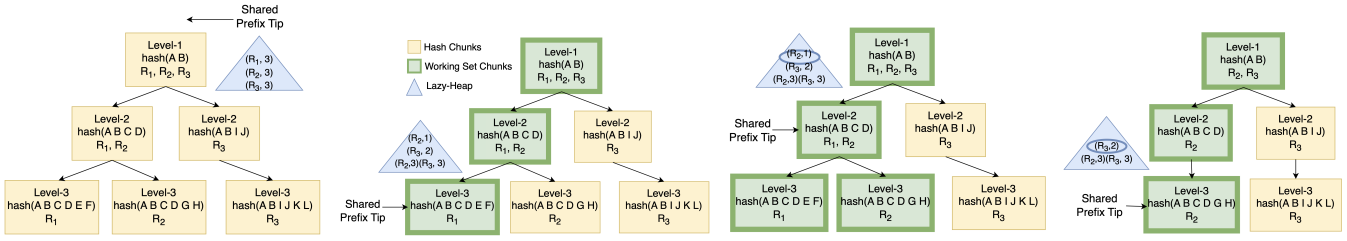
System Architecture. Figure 10 shows the overall scheduling pipeline. We consider an inference server that continuously receives requests, which are inserted into the waiting queue of CHT. The scheduler builds batches incrementally in a loop. It first finds the best candidate by retrieving the waiting request with the highest overlap with the current working set. Then the RL policy evaluates the current system state and decides whether to Add the request or Stop. If Add is chosen, the request is added to the scheduler batch, and all the data structures, such as the working set and the min-heap, are updated accordingly; otherwise, the batch is dispatched to the GPU for an execution step. The decode throughput measured after the GPU execution is used as a reward for the RL policy. Completed requests are removed from the active batch, and the working set is updated.

4.2 Chunked Hash Tree (CHT)

We now describe the operations of CHT (a more formal treatment is in §C). We begin with how token sequences are mapped to a compact representation in our system.

Hash Vectors. Given a request as a token sequence, we partition it into non-overlapping chunks of size K and build a sequence of hashes, one for each chunk, where each hash is cumulative over all tokens up to that chunk. For example, in Figure 11(a), a request with tokens A-F and $K = 2$ is represented as three incremental hashes: hash(A-B), hash(A-D), and hash(A-F). Two requests will have the same hash at level l iff they share the same first lK tokens. Each request is represented as an ordered hash vector. We use an incremental xxHash-64 state, ensuring each token is processed once, yielding a total cost of $O(T)$, independent of the number of chunk boundaries, where T is the sequence length.

Insertion. Insert is invoked when a new request arrives at the scheduler. We first compute the cumulative prefix-hash vector and store it in a map from request IDs to hash vectors for faster reuse. We also maintain a reverse index that maps each hash to the set of requests containing that hash. For



(a) Insertion **(b)** Adding to Batch **(c)** Finding the Best Request **(d)** Request Completion

Figure 11. Chunked Hash Tree Operations

each hash in the hash vector, we insert the request into this reverse index. The overall cost is dominated by the hash computation phase, which runs in $O(T)$.

Working Set, Min-Heap and Shared Prefix Tip. To evaluate how well a request matches the active batch, we maintain a working set that contains (level, hash) tuples representing prefix hashes at that level covered by currently active requests. For each request, we also maintain a missing count, which is the number of levels for which this request's hashes are not present in the working set. We insert these missing counts into a min-heap, which enables efficient retrieval of the most suitable request for addition to the batch. INSERT updates both of these structures. Moreover, we do not perform in-place updates and use lazy deletion to discard stale entries in the heap. To relate to our key metric, which is the number of chunks shared by all active requests, we also maintain a shared prefix tip, storing the depth and hash of the deepest node that is shared by all the active requests. Figure 11(b) shows the Chunked Hash Tree after the request R_1 is added to the batch. The nodes marked in green denote the working set. Since there is just a single request, the shared prefix tip is at Level-3 of R_1. The heap contains the missing counts of the waiting requests along with stale entries.

Finding the Best Request. The goal of FINDBEST is to select the waiting request that best complements the active batch, i.e., the one with the minimum missing count, so that we maximize the depth of the shared prefix tip[2]. Since the min-heap is keyed by this count, the best candidate appears at the top after skipping stale entries. As shown in Figure 11(c), R_2 appears at the top of the min-heap and is therefore selected as the best request. Note that adding R_2 to the batch reduces the shared prefix tip by one level. FINDBEST runs in $O(\log W)$, where W is the number of waiting requests. For the selected request, we compute two metrics that are inputs to the RL policy (§4.3), which needs to decide whether adding the request to the current batch is beneficial for the overall throughput. One of the metrics is the change in the depth of the shared prefix tip, and the other is the number of additional waiting requests that are present at this new depth, which can all be added to the batch without any further impact on the shared prefix tip in the future.

Adding a Request to the Active Batch. When a request is promoted to the active batch, we update the working set to include its prefixes and recompute the shared prefix tip. We iterate over the hash vector backwards; for each level, if the corresponding prefix hash isn't present in the working set, it is inserted, and all waiting requests sharing it have their missing count decremented. For the shared prefix tip, admitting a new request can only *shorten* the common prefix, so we seek the deepest level where it agrees with the existing tip hash. This corresponds to the least common ancestor of all active request leaves in the hash trie. ADDTOBATCH runs in $O(C \cdot W_h \cdot \log W)$ time, where $C = \lceil T/K \rceil$ is the number of chunks for a request with T tokens, and W_h is the number of waiting requests that share the same hash ($W_h \ll W$).

Removing a Request from the Active Batch. FINISH is called when a request completes, and we remove it from the active batch. It is the symmetric counterpart of ADDTOBATCH. We iterate over the hash vector from the start. A hash unused by any active request is removed from the working set, and all waiting requests containing it have their missing count incremented. A request completion can only *lengthen* the shared prefix tip since the remaining active requests may agree on a longer common prefix. Starting from the tip's previous position, we scan forward, checking whether all remaining active requests share the same hash at each level. (We do this efficiently by maintaining reference counts of each hash.) The extension stops at the first level of disagreement or at the shortest active request. This scan is inexpensive in practice, as the tip rarely advances more than a few levels. FINISH also runs in $O(C \cdot W_h \cdot \log W)$ time. Figure 11(d) shows the state after R_1 completes: the tip extends to R_2's final level, and waiting heap costs are updated accordingly.

Optimizations. We employ several optimizations to improve efficiency. First, FINDBEST may be invoked multiple times within a scheduling round without state changes, so we cache its last result and reuse it until it is invalidated by any mutation (e.g., ADDTOBATCH and FINISH). Second, we use lazy heap updates: when a request's missing count changes, we push a new entry instead of updating it in place and skip stale entries during FINDBEST. Third, we maintain

[2]An astute reader might ask why we are choosing a request based on its difference from the working set rather than its impact on the length of the shared prefix tip. A brief explanation is that we want to pick a request that helps us revert back to a longer shared prefix tip sooner, and a request that differs from the working set by the least number of chunks helps us achieve this goal best. We refer the reader to Appendix C.9 for more details.

the shared prefix tip incrementally rather than recomputing it. This drops the cost from $O(A \cdot C)$, where A is the number of active requests, and C is the number of chunks, to just $O(C)$ in the worst case and typically terminates earlier. Finally, all major data structures are pre-allocated at initialization to eliminate rehashing and reallocation overhead.

4.3 Batching and Reinforcement Learning

The CHT efficiently identifies the next best request to add, maximizing shared prefix length, but it does not determine *when to stop*. While adding requests improves GPU utilization, it can also shrink the shared prefix and degrade locality. We frame this stop/continue decision as a learning problem and present three policies: a heuristic, a contextual bandit, and a Q-learning agent. The scheduler incrementally builds a batch by selecting candidates via the CHT and deciding whether to admit them or dispatch the batch.

State, Actions, and Reward. At each step of batch formation, the scheduler observes the state $s = (b,\ \Delta,\ w)$ and takes an action $a \in \{\textsc{Add}, \textsc{Stop}\}$. Here, b is the current batch size (a proxy for GPU utilization), Δ is the loss in the shared prefix, and w is the number of waiting requests sharing the prefix up to the new depth, indicating whether the locality cost of this addition will be compensated by future activations. Both RL policies optimize the reward $\mathcal{R} = \text{throughput}_{\text{decode}}$. Maximizing this encourages large batches and short decode times (better prefix locality). The state is discretized into $(\hat{b}, \hat{\Delta}, \hat{w})$, where b and w use exponential bins, and Δ uses coarse thresholds. The resulting table is small, hashable, and converges quickly under moderate load.

Heuristic Policy. As a baseline, we encode domain intuition as simple rules balancing two goals: maximizing GPU utilization (large batches) and preserving prefix locality (avoiding large drops in the shared chain). The heuristic always accepts candidates with zero chunk loss (free additions). When the batch is small, it tolerates higher loss to avoid underutilization. Otherwise, it admits a candidate only if the chunk loss is within a fixed threshold, with slightly relaxed tolerance when many waiting requests share the resulting prefix (anticipating amortization). If none of these conditions hold, it stops and dispatches the batch. This policy requires no training and converges immediately, but its fixed thresholds cannot adapt to workload or hardware changes.

Contextual Bandit. To avoid hand-tuned thresholds, we use a contextual bandit with Upper Confidence Bound (UCB) selection [5]. Each batch-construction step is treated as an independent decision: given the discretized state $\hat{s} = (\hat{b}, \hat{\Delta}, \hat{w})$, the policy selects the action that maximizes

$$\text{UCB}(\hat{s}, a) = \frac{\mu(\hat{s}, a)}{n(\hat{s}, a)} + c\sqrt{\frac{\ln S}{n(\hat{s}, a)}}.$$

Here, $\mu(\hat{s}, a)$ is the cumulative reward, $n(\hat{s}, a)$ is the number of times action a was taken in $\hat{s}$, S is the total number of decisions, and c is the exploration coefficient. The first term favors high-reward actions, while the second promotes exploration of under-sampled ones.

Q-Learning Policy. As an alternative to the previous policy, we can also model batch construction as an episodic reinforcement learning problem, where each episode begins when forming a new batch and ends upon Stop or when the waiting queue is exhausted. The Q-table [35] stores values $Q(s, a)$ for each (discretized state, action) pair and is updated after each episode using:

$$Q(s, a) \ \leftarrow\ Q(s, a) + \alpha\big[r + \gamma \max_{a'} Q(s', a') - Q(s, a)\big].$$

Here, r is the observed throughput, s' is the next state, α the learning rate, and γ the discount factor, which biases the agent toward near-term throughput, consistent with real-time inference. We follow an ε-greedy exploration strategy: at each step, a random action is selected with probability ε, and the greedy action $\arg\max_a Q(s, a)$ otherwise. During training, ε decays from 1.0 to $\varepsilon_{\min}$, transitioning from full exploration to near-deterministic behavior. Qualitatively, we expect this policy to perform better in the long term when actions influence future batch composition.

At each step, FindBest returns the lowest-cost candidate along with (Δ, w), forming the state s. The policy (heuristic, bandit, or Q-learning) then selects Add or Stop. If Add, the request is activated and appended to the batch; otherwise, the batch is dispatched. After execution, the observed decode throughput is used as the reward: the bandit updates its statistics and the Q-learning agent applies the Bellman update. This has also been shown in Figure 10.

4.4 Integration with vLLM and SGLang

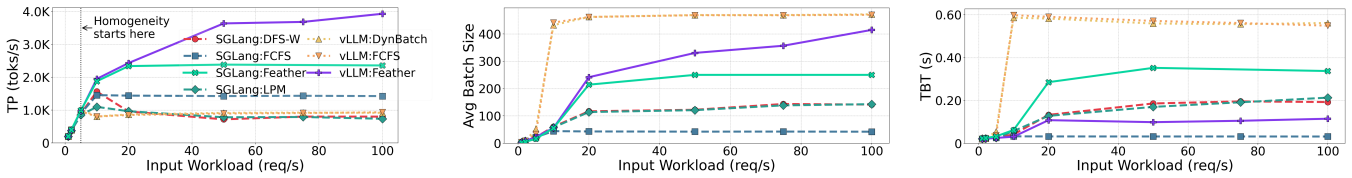
We integrate our scheduler as a lightweight layer on top of both vLLM [17] and SGLang [44]. The Chunked Hash Tree (implemented in C++) along with shared prefix chain tracking and reinforcement learning policies operates entirely at the scheduler level. We will upstream our changes to these repositories upon acceptance of our publication.

5 Evaluation

We evaluate Feather across diverse models, GPU configurations, and real-world datasets and scenarios. Our evaluation answers the following questions: (i) Under what workloads do smaller homogeneous batches outperform larger heterogeneous ones? (ii) How effective is Feather's Chunked Hash Tree compared to traditional radix-tree-based approaches? (iii) How do different design components of Feather (RL, CHT) interact to produce end-to-end performance gains?

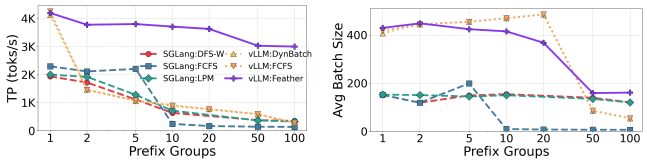
5.1 Experimental Setup

Models and Hardware. We evaluate three model families across multiple sizes: Qwen 0.5B, 1.5B, 8B [6], LLaMA 3 8B [2], and LongChat 13B [20]. Experiments are run on an NVIDIA RTX 6000 Ada GPU [23] with 48 GB GDDR6, 96 MB L2, and the maximum batch size is fixed at 500, which fits within the GPU memory for most settings. We additionally



(a) Throughput **(b)** Average Batch Size **(c)** Time between tokens (TBT)

Figure 12. Poisson Workload



(a) Throughput **(b)** Average Batch Size

Figure 13. Varying Number of Prefix Groups

evaluate on an A100-80GB GPU for comparisons involving PAT [41], as it is specifically optimized for this hardware.

Workloads. Following prior work, request arrivals are modeled as a Poisson process, with length pruning applied as required by each experiment. L-Eval [3] is a benchmark of human-labeled query–response pairs spanning tasks such as summarization and question answering, with sequence lengths ranging from 2.7K to 210.5K tokens. LongBench [7] is a collection of long-context samples across six task categories: multi-document QA, summarization, and code completion, with context lengths between 4K and 10K tokens.

Baselines. We compare Feather against state-of-the-art LLM serving systems, all with prefix caching enabled. vLLM [17] uses its default FCFS scheduling policy. SGLang [44] is evaluated under FCFS, LPM, and DFS-W to cover its design space. We also evaluate DynamicBatching [28] and PAT [41], both implemented on top of vLLM: DynamicBatching adjusts batch sizes based on runtime memory utilization and latency constraints, while PAT is a prefix-aware attention kernel that reduces redundant KV cache accesses via query packing.

Metrics. We report three key metrics. Throughput (toks/s) measures the output tokens generated per second, reflecting serving capacity and GPU utilization. Time between tokens (TBT) measures the average interval between decoded tokens, reflecting responsiveness, especially in interactive settings. We also track the average batch size per forward pass to relate scheduling decisions to these metrics.

5.2 End-to-End Comparison: Feather vs Baselines

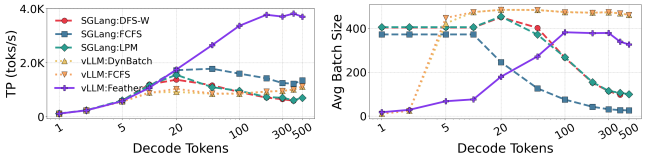
We evaluate a workload where the default configuration has 5 prefix groups[3], each 5K tokens long, with roughly equal request distribution, a request rate of 100 req/s, and 200 output tokens per request. Such workload configurations are common in prior work [27, 41] evaluating LLM serving systems. We vary these parameters to examine how Feather compares to the baselines. The token budget is 32K tokens.

[3]We also evaluate workloads without prefix sharing and show Feather performs on par with FCFS (§D.7).

For Feather's RL policy, we find that bandit and Q-learning approaches perform similarly across most settings; unless stated otherwise, we report results using the bandit policy.

Varying Input Request Rate. Figure 12(a) shows throughput (toks/s) as a function of the request rate. Across all policies, throughput increases with the request rate before saturating. Feather (on vLLM) consistently outperforms all baselines at every request rate. At 100 req/s, Feather achieves roughly 4× the throughput of vLLM (FCFS). Notably, this advantage is not driven by larger batch sizes. As shown in Figure 12(b), Feather in general maintains a *smaller* average batch size than vLLM (FCFS), and beyond the vertical line shown in the plot, Feather's RL policy decides to have completely homogeneous batches. This reinforces a key insight: *smaller, homogeneous batches outperform larger, heterogeneous ones.* The gradual growth in Feather's batch size with increasing request rate reflects its ability to efficiently aggregate more requests from the same prefix group into the active batch without sacrificing locality. SGLang policies consistently operate at smaller batch sizes than vLLM, as vLLM avoids double-counting shared prefix tokens toward the token budget, allowing more requests per batch. Dynamic Batching performs similarly to vLLM (FCFS), as all KV caches fit within GPU memory. Figure 12(c) shows time between tokens (TBT). Feather (on vLLM) achieves substantially lower TBT than all baselines except SGLang (FCFS), whose smaller batch sizes yield faster per-token generation. While SGLang's LPM and DFS-W policies also use small batch sizes, their TBT remains significantly higher than Feather due to non-trivial scheduling overhead during batch construction. Feather implemented on SGLang also shows similar qualitative trends and outperforms all other SGLang policies. Unless otherwise stated, all subsequent results refer to Feather implemented on vLLM.

Varying Number of Prefix Groups. Figures 13(a) and 13(b) show throughput and average batch size as functions of the number of prefix groups. Increasing the number of prefix groups introduces greater request heterogeneity, reducing KV cache locality. This leads to a sharp throughput drop for vLLM (FCFS), even when increasing from 1 to 2 prefix groups. As the number of groups increases further, FCFS degrades rapidly: beyond 20 groups, the KV cache footprint exceeds HBM capacity, causing a sharp reduction in batch size and increased recomputation. Feather mitigates this by grouping requests with shared prefixes, increasing KV cache



(a) Throughput **(b)** Average Batch Size

Figure 14. Varying Number of Decode Tokens

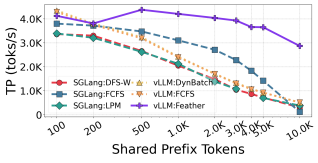


Figure 15. Varying Shared Prefix Tokens

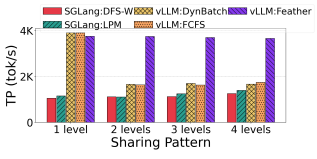


Figure 16. Varying Radix Tree Sharing Levels

reuse and avoiding premature evictions. As a result, FEATHER maintains a more stable batch size and throughput across all prefix group configurations, significantly outperforming all baselines. At 100 prefix groups, FEATHER achieves 10× higher throughput than vLLM (FCFS). SGLang's LPM and DFS-W policies partially mitigate evictions, achieving higher throughput than FCFS (SGLang) at larger group counts.

Varying Number of Decode Tokens. Figures 14(a) and 14(b) show throughput and average batch size as functions of decode tokens per request. Each decode requires a full sweep over the KV cache. The policies exhibit two regimes separated by a crossover around 20 decode tokens. In the prefill-dominated regime (1–20 tokens), all policies behave similarly: throughput increases modestly, and batch sizes converge as prefills limit the impact of locality-aware scheduling. Beyond 20 tokens, the policies diverge sharply. FEATHER scales strongly with decode length, reaching ~3800 toks/s at 200 tokens—over 3× higher than SGLang policies. This gain arises from two factors. First, longer decode phases amplify locality across repeated KV cache sweeps. Second, a greater number of decode iterations allows the scheduler to accumulate more same-prefix requests, forming larger, more homogeneous batches. Together, these effects drive the steady increase in FEATHER's batch size and throughput. SGLang policies follow the opposite trajectory in batch size. Because shared prefix tokens count toward the token budget, each prefill consumes ~5K tokens, while each decode step contributes only one token. As the decode length increases, fewer prefills occur per step, reducing the number of new requests for batching.

Varying Number of Shared Prefix Tokens. Figure 15 shows throughput across the length of the shared prefix. As expected, longer prefixes increase prefill compute and KV cache pressure, reducing throughput across policies. FEATHER's advantage grows with prefix length, driven by higher locality gains as sequences lengthen.

Varying Models. FEATHER consistently achieves the highest throughput across models and outperforms baselines in nearly all cases. Its relative throughput advantage increases

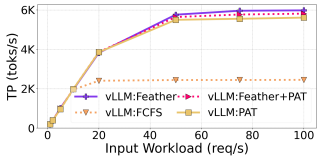


Figure 17. FEATHER vs PAT

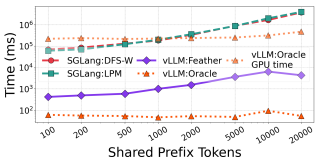


Figure 18. CPU overhead

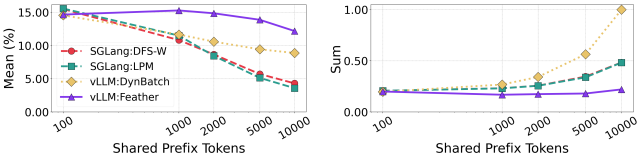
with model size, as larger models spend more time in attention, amplifying the benefits of KV-cache locality exploited by FEATHER. All graphs are shown in §D.2. We also compare FEATHER with other policies (§D.3) for the LongChat 13B model, where we achieve up to 22× higher throughput compared to vLLM's FCFS for larger number of prefix groups.

Varying Radix Tree Levels. Figure 16 shows throughput under different radix-tree sharing levels (seen in §3.1). All experiments use a fixed total sequence length of 4000 tokens per request. For baseline policies like vLLM's FCFS, going from 1 to 2 or more levels of the radix tree reduces the locality in KV cache traversal and, hence, degrades throughput. In contrast, FEATHER actively preserves prefix homogeneity within batches, leading to stable throughput at all levels.

Comparison with PAT. Figure 17 compares throughput across varying request rates for PAT, vLLM (FCFS), and FEATHER on an A100-80GB GPU. PAT is specifically optimized for this hardware through custom prefix-aware kernels. Despite this, FEATHER consistently matches or outperforms PAT across all request rates, demonstrating strong performance even on different hardware. This is particularly notable because FEATHER operates purely at the scheduling layer, without any kernel-level modifications. A naive integration of FEATHER and PAT performs quite similarly to FEATHER over most regimes. As future work, we aim to more tightly integrate FEATHER with prefix-aware attention kernels, enabling deeper co-design between the scheduler and kernel-level optimizations to further improve performance.

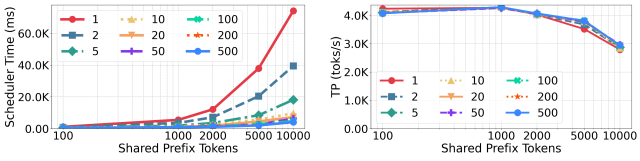
5.3 Micro-Benchmarks

Comparing Scheduler Compute Overhead across Different Policies. Figure 18 shows the CPU scheduling overhead for each policy across different shared prefix lengths. We include an Oracle policy where the caller supplies prefix group identifiers with each request. This allows the scheduler to batch same-prefix requests without any prefix matching overhead, resulting in effectively zero scheduling cost while preserving locality. We also show the GPU execution time for this Oracle scheduling policy. All three realistic policies—FEATHER, LPM, and DFS-W— incur higher overhead as the prefix length increases due to deeper radix-tree traversals and larger key comparisons. However, their growth rates diverge significantly. At 20K tokens, LPM and DFS-W incur roughly 1,000× higher overhead than FEATHER. At larger sequence lengths, their scheduling overhead exceeds the Oracle GPU execution time by up to 10×. This indicates that CPU overhead alone can negate the benefits of cache-aware



(a) Mean **(b)** Sum

Figure 19. DRAM Bandwidth Utilization



(a) CPU Scheduler Overhead **(b)** Throughput

Figure 20. Sensitivity to Chunk Size

batching. In contrast, FEATHER remains below 1% of GPU execution time across the entire range. We do a detailed analysis of the overheads of LPM and DFS-W policies in §D.4.

Mean DRAM Activity across Various Sequence Lengths. Figure 19(a) shows mean DRAM bandwidth utilization, while Figure 19(b) shows the total DRAM usage over the full inference duration (a proxy for total memory accessed) across policies and shared prefix lengths. FEATHER consistently achieves the highest bandwidth utilization due to improved spatial locality, leading to more effective prefetching. Furthermore, more homogeneous batches also increase temporal locality, which is reflected in lower total DRAM usage. SGLang's DFS-W and LPM exhibit lower mean DRAM utilization because of CPU scheduler stalls. However, they also achieve lower normalized total DRAM usage, as their prefix-aware scheduling occasionally forms homogeneous batches, allowing them to partially benefit from memory locality.

Effect of Chunk Size. Figures 20(a) and 20(b) show CPU overhead and throughput across shared prefix lengths for different chunk sizes in FEATHER's CHT. Increasing the chunk size reduces CPU overhead by enabling coarser-grained matching; a chunk size of 1 is equivalent to an exact radix tree. Overhead rises with prefix length due to more hash comparisons, while throughput declines due to higher KV cache memory pressure. At larger prefix lengths, the overhead gap between chunk size 1 and larger sizes widens substantially, yet throughput remains similar across all chunk sizes, suggesting that the CPU scheduling cost, even at the finest granularity, is not the primary bottleneck in our CHT. Overall, CHT is robust against chunk size selection, and even its radix tree-equivalent configuration (chunk size 1) achieves significantly better efficiency than a conventional radix tree.

5.4 Ablation Study

What if we Replace the Chunked Hash Tree with a Radix Tree? Figure 21 compares throughput and TBT when replacing CHT with a radix tree under the same RL policy. Across all prefix lengths, the radix tree yields much lower throughput. Despite homogeneous RL batching, higher CPU

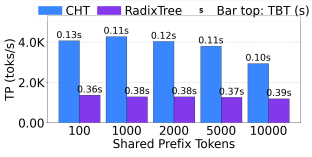


Figure 21. CHT vs. Radix Tree

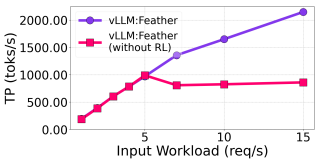


Figure 22. FEATHER: with and without RL

overhead causes frequent GPU stalls. This appears as higher TBT, indicating GPU idle time due to scheduling delays. CHT avoids this by reducing scheduling overhead, enabling higher GPU utilization and throughput.

What if We Remove RL?. Figure 22 compares throughput with and without RL at low input rates. Without RL, CHT still selects the best request, but batching never halts and always maximizes batch size. At <5 req/s, removing RL has little effect. Both fall back to FCFS, adding requests to keep GPUs utilized. Beyond this, the behaviors diverge. With RL, FEATHER detects homogeneous requests and stops batching at the right time, achieving higher throughput despite smaller average batch sizes. At higher rates than shown in the figure, large pools enable even non-RL to form ~500-request homogeneous batches, leading the two curves to converge. These results show that FEATHER's RL policy makes load-aware decisions: it falls back to FCFS under low utilization to avoid GPU starvation, and closes batch formation when homogeneity is achievable, maximizing overall throughput.

Interested readers can see §D for additional results.

6 Conclusion

In this paper, we showed that maximizing batch size alone is not sufficient for efficient LLM serving during the decode phase. Instead, we identified *prefix homogeneity*, the extent of the prefix shared across *all* requests in a batch, as a more important factor for improving memory locality and reducing redundant KV cache accesses. Higher batch sizes lead to better GPU utilization but also reduce the length of the prefix shared across all requests. Existing approaches ignore this aspect and also rely on expensive prefix detection mechanisms that introduce significant CPU overhead. To address these gaps, we proposed FEATHER, a lightweight prefix-aware scheduler that balances this trade-off between batch size and prefix homogeneity. It uses a Chunked Hash Tree (CHT) for fast prefix detection and a reinforcement learning-based batching policy that decides when to stop batch formation based on the loss in prefix homogeneity due to the addition of a new request. Our results show that FEATHER consistently improves end-to-end throughput across diverse workloads while keeping scheduling overhead low. As future work, we plan to extend this approach to distributed and multi-GPU settings. We are also looking into more adaptive and lightweight learning policies that can improve robustness across different workloads and deployment scenarios.

References

[1] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming throughput-latency tradeoff in LLM inference with sarathi-serve. In Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation. USENIX Association.

[2] AI@Meta. 2024. Llama 3 Model Card. (2024). https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md

[3] Chenxin An, Shansan Gong, Ming Zhong, Xingjian Zhao, Mukai Li, Jun Zhang, Lingpeng Kong, and Xipeng Qiu. 2024. L-Eval: Instituting Standardized Evaluation for Long Context Language Models. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). Association for Computational Linguistics. doi:10.18653/v1/2024.acl-long.776

[4] Anthropic. 2024. Claude. https://claude.ai.

[5] Peter Auer, Nicolò Cesa-Bianchi, and Paul Fischer. 2002. Finite-time Analysis of the Multiarmed Bandit Problem. Machine Learning (2002). doi:10.1023/A:1013689704352

[6] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, Binyuan Hui, Luo Ji, Mei Li, Junyang Lin, Runji Lin, Dayiheng Liu, Gao Liu, Chengqiang Lu, Keming Lu, Jianxin Ma, Rui Men, Xingzhang Ren, Xuancheng Ren, Chuanqi Tan, Sinan Tan, Jianhong Tu, Peng Wang, Shijie Wang, Wei Wang, Shengguang Wu, Benfeng Xu, Jin Xu, An Yang, Hao Yang, Jian Yang, Shusheng Yang, Yang Yao, Bowen Yu, Hongyi Yuan, Zheng Yuan, Jianwei Zhang, Xingxuan Zhang, Yichang Zhang, Zhenru Zhang, Chang Zhou, Jingren Zhou, Xiaohuan Zhou, and Tianhang Zhu. 2023. Qwen Technical Report. arXiv preprint arXiv:2309.16609 (2023).

[7] Yushi Bai, Shangqing Tu, Jiajie Zhang, Hao Peng, Xiaozhi Wang, Xin Lv, Shulin Cao, Jiazheng Xu, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. 2025. LongBench v2: Towards Deeper Understanding and Reasoning on Realistic Long-context Multitasks. In Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). Association for Computational Linguistics. doi:10.18653/v1/2025.acl-long.183

[8] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language models are few-shot learners. In Proceedings of the 34th International Conference on Neural Information Processing Systems. Curran Associates Inc.

[9] Jerry Chee, Yaohui Cai, Volodymyr Kuleshov, and Christopher De Sa. 2023. QuIP: 2-bit quantization of large language models with guarantees. In Proceedings of the 37th International Conference on Neural Information Processing Systems. Curran Associates Inc.

[10] Sumit Kumar Dam, Choong Seon Hong, Yu Qiao, and Chaoning Zhang. 2024. A Complete Survey on LLM-based AI Chatbots. arXiv:2406.16937 [cs.CL] https://arxiv.org/abs/2406.16937

[11] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FLASHATTENTION: fast and memory-efficient exact attention with IO-awareness. In Proceedings of the 36th International Conference on Neural Information Processing Systems. Curran Associates Inc.

[12] DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Haowei Zhang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Li, Hui Qu, J. L. Cai, Jian Liang, Jianzhong Guo, Jiaqi Ni, Jiashi Li, Jiawei Wang, Jin Chen, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, Junxiao Song, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Lei Xu, Leyi Xia, Liang Zhao, Litong Wang, Liyue Zhang, Meng Li, Miaojun Wang, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingming Li, Ning Tian, Panpan Huang, Peiyi Wang, Peng Zhang, Qiancheng Wang, Qihao Zhu, Qinyu Chen, Qiushi Du, R. J. Chen, R. L. Jin, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runxin Xu, Ruoyu Zhang, Ruyi Chen, S. S. Li, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shaoqing Wu, Shengfeng Ye, Shengfeng Ye, Shirong Ma, Shiyu Wang, Shuang Zhou, Shuiping Yu, Shunfeng Zhou, Shuting Pan, T. Wang, Tao Yun, Tian Pei, Tianyu Sun, W. L. Xiao, Wangding Zeng, Wanjia Zhao, Wei An, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, X. Q. Li, Xiangyue Jin, Xianzu Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaojin Shen, Xiaokang Chen, Xiaokang Zhang, Xiaosha Chen, Xiaotao Nie, Xiaowen Sun, Xiaoxiang Wang, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xingkai Yu, Xinnan Song, Xinxia Shan, Xinyi Zhou, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. X. Zhu, Yang Zhang, Yanhong Xu, Yanhong Xu, Yanping Huang, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Li, Yaohui Wang, Yi Yu, Yi Zheng, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Ying Tang, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yu Wu, Yuan Ou, Yuchen Zhu, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yukun Zha, Yunfan Xiong, Yunxian Ma, Yuting Yan, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Z. F. Wu, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhen Huang, Zhen Zhang, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhibin Gou, Zhicheng Ma, Zhigang Yan, Zhihong Shao, Zhipeng Xu, Zhiyu Wu, Zhongyu Zhang, Zhuoshu Li, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Ziyi Gao, and Zizheng Pan. 2025. DeepSeek-V3 Technical Report. arXiv:2412.19437 [cs.CL] https://arxiv.org/abs/2412.19437

[13] Xin Luna Dong, Seungwhan Moon, Yifan Ethan Xu, Kshitiz Malik, and Zhou Yu. 2023. Towards Next-Generation Intelligent Assistants Leveraging LLM Techniques. In Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining. Association for Computing Machinery. doi:10.1145/3580305.3599572

[14] Google. 2024. Gemini. https://gemini.google.com.

[15] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, Kai Dang, Yang Fan, Yichang Zhang, An Yang, Rui Men, Fei Huang, Bo Zheng, Yibo Miao, Shanghaoran Quan, Yunlong Feng, Xingzhang Ren, Xuancheng Ren, Jingren Zhou, and Junyang Lin. 2024. Qwen2.5-Coder Technical Report. arXiv:2409.12186 [cs.CL] https://arxiv.org/abs/2409.12186

[16] Aditya K. Kamath, Ramya Prabhu, Jayashree Mohan, Simon Peter, Ramachandran Ramjee, and Ashish Panwar. 2025. POD-Attention: Unlocking Full Prefill-Decode Overlap for Faster LLM Inference. In Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2. Association for Computing Machinery. doi:10.1145/3676641.3715996

[17] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In Proceedings of the 29th Symposium on Operating Systems Principles. Association for Computing Machinery. doi:10.1145/3600006.3613165

[18] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In Advances in Neural Information Processing Systems. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2020/

file/6b493230205f780e1bc26945df7481e5-Paper.pdf

[19] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In Proceedings of the 34th International Conference on Neural Information Processing Systems. Curran Associates Inc.

[20] Dacheng Li, Rulin Shao, Anze Xie, Ying Sheng, Lianmin Zheng, Joseph E. Gonzalez, Ion Stoica, Xuezhe Ma, and Hao Zhang. 2023. How Long Can Open-Source LLMs Truly Promise on Context Length? https://lmsys.org/blog/2023-06-29-longchat

[21] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Guangxuan Xiao, and Song Han. 2025. AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration. (2025). doi:10.1145/3714983.3714987

[22] Zejia Lin, Hongxin Xu, Guanyi Chen, Zhiguang Chen, Yutong Lu, and Xianwei Zhang. 2026. Bullet: Boosting GPU Utilization for LLM Serving via Dynamic Spatial-Temporal Orchestration. In Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2. Association for Computing Machinery. doi:10.1145/3779212.3790135

[23] NVIDIA Corporation. 2024. NVIDIA RTX 6000 Ada Generation Datasheet. https://resources.nvidia.com/en-us-briefcase-for-datasheets/proviz-print-rtx6000-1?ncid=no-ncid Accessed: January 25, 2026.

[24] NVIDIA Corporation. 2026. NVIDIA Data Center GPU Manager (DCGM) Documentation. https://docs.nvidia.com/datacenter/dcgm/latest/user-guide/.

[25] OpenAI. 2024. ChatGPT. https://chatgpt.com.

[26] OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, Red Avila, Igor Babuschkin, Suchir Balaji, Valerie Balcom, Paul Baltescu, Haiming Bao, Mohammad Bavarian, Jeff Belgum, Irwan Bello, Jake Berdine, Gabriel Bernadett-Shapiro, Christopher Berner, Lenny Bogdonoff, Oleg Boiko, Madelaine Boyd, Anna-Luisa Brakman, Greg Brockman, Tim Brooks, Miles Brundage, Kevin Button, Trevor Cai, Rosie Campbell, Andrew Cann, Brittany Carey, Chelsea Carlson, Rory Carmichael, Brooke Chan, Che Chang, Fotis Chantzis, Derek Chen, Sully Chen, Ruby Chen, Jason Chen, Mark Chen, Ben Chess, Chester Cho, Casey Chu, Hyung Won Chung, Dave Cummings, Jeremiah Currier, Yunxing Dai, Cory Decareaux, Thomas Degry, Noah Deutsch, Damien Deville, Arka Dhar, David Dohan, Steve Dowling, Sheila Dunning, Adrien Ecoffet, Atty Eleti, Tyna Eloundou, David Farhi, Liam Fedus, Niko Felix, Simón Posada Fishman, Juston Forte, Isabella Fulford, Leo Gao, Elie Georges, Christian Gibson, Vik Goel, Tarun Gogineni, Gabriel Goh, Rapha Gontijo-Lopes, Jonathan Gordon, Morgan Grafstein, Scott Gray, Ryan Greene, Joshua Gross, Shixiang Shane Gu, Yufei Guo, Chris Hallacy, Jesse Han, Jeff Harris, Yuchen He, Mike Heaton, Johannes Heidecke, Chris Hesse, Alan Hickey, Wade Hickey, Peter Hoeschele, Brandon Houghton, Kenny Hsu, Shengli Hu, Xin Hu, Joost Huizinga, Shantanu Jain, Shawn Jain, Joanne Jang, Angela Jiang, Roger Jiang, Haozhun Jin, Denny Jin, Shino Jomoto, Billie Jonn, Heewoo Jun, Tomer Kaftan, Łukasz Kaiser, Ali Kamali, Ingmar Kanitscheider, Nitish Shirish Keskar, Tabarak Khan, Logan Kilpatrick, Jong Wook Kim, Christina Kim, Yongjik Kim, Jan Hendrik Kirchner, Jamie Kiros, Matt Knight, Daniel Kokotajlo, Łukasz Kondraciuk, Andrew Kondrich, Aris Konstantinidis, Kyle Kosic, Gretchen Krueger, Vishal Kuo, Michael Lampe, Ikai Lan, Teddy Lee, Jan Leike, Jade Leung, Daniel Levy, Chak Ming Li, Rachel Lim, Molly Lin, Stephanie Lin, Mateusz Litwin, Theresa Lopez, Ryan Lowe, Patricia Lue, Anna Makanju, Kim Malfacini, Sam Manning, Todor Markov, Yaniv Markovski, Bianca Martin, Katie Mayer, Andrew Mayne, Bob McGrew, Scott Mayer McKinney, Christine McLeavey, Paul McMillan, Jake McNeil, David Medina, Aalok Mehta, Jacob Menick, Luke Metz, Andrey Mishchenko, Pamela Mishkin, Vinnie Monaco, Evan Morikawa, Daniel Mossing, Tong Mu, Mira Murati, Oleg Murk, David Mély, Ashvin Nair, Reiichiro Nakano, Rajeev Nayak, Arvind Neelakantan, Richard Ngo, Hyeonwoo Noh, Long Ouyang, Cullen O'Keefe, Jakub Pachocki, Alex Paino, Joe Palermo, Ashley Pantuliano, Giambattista Parascandolo, Joel Parish, Emy Parparita, Alex Passos, Mikhail Pavlov, Andrew Peng, Adam Perelman, Filipe de Avila Belbute Peres, Michael Petrov, Henrique Ponde de Oliveira Pinto, Michael, Pokorny, Michelle Pokrass, Vitchyr H. Pong, Tolly Powell, Alethea Power, Boris Power, Elizabeth Proehl, Raul Puri, Alec Radford, Jack Rae, Aditya Ramesh, Cameron Raymond, Francis Real, Kendra Rimbach, Carl Ross, Bob Rotsted, Henri Roussez, Nick Ryder, Mario Saltarelli, Ted Sanders, Shibani Santurkar, Girish Sastry, Heather Schmidt, David Schnurr, John Schulman, Daniel Selsam, Kyla Sheppard, Toki Sherbakov, Jessica Shieh, Sarah Shoker, Pranav Shyam, Szymon Sidor, Eric Sigler, Maddie Simens, Jordan Sitkin, Katarina Slama, Ian Sohl, Benjamin Sokolowsky, Yang Song, Natalie Staudacher, Felipe Petroski Such, Natalie Summers, Ilya Sutskever, Jie Tang, Nikolas Tezak, Madeleine B. Thompson, Phil Tillet, Amin Tootoonchian, Elizabeth Tseng, Preston Tuggle, Nick Turley, Jerry Tworek, Juan Felipe Cerón Uribe, Andrea Vallone, Arun Vijayvergiya, Chelsea Voss, Carroll Wainwright, Justin Jay Wang, Alvin Wang, Ben Wang, Jonathan Ward, Jason Wei, CJ Weinmann, Akila Welihinda, Peter Welinder, Jiayi Weng, Lilian Weng, Matt Wiethoff, Dave Willner, Clemens Winter, Samuel Wolrich, Hannah Wong, Lauren Workman, Sherwin Wu, Jeff Wu, Michael Wu, Kai Xiao, Tao Xu, Sarah Yoo, Kevin Yu, Qiming Yuan, Wojciech Zaremba, Rowan Zellers, Chong Zhang, Marvin Zhang, Shengjia Zhao, Tianhao Zheng, Juntang Zhuang, William Zhuk, and Barret Zoph. 2024. GPT-4 Technical Report. arXiv:2303.08774 [cs.CL] https://arxiv.org/abs/2303.08774

[27] Zaifeng Pan, Yitong Ding, Yue Guan, Zheng Wang, Zhongkai Yu, Xulong Tang, Yida Wang, and Yufei Ding. 2025. FastTree: Optimizing Attention Kernel and Runtime for Tree-Structured LLM Inference. In Proceedings of Machine Learning and Systems. MLSys. https://proceedings.mlsys.org/paper_files/paper/2025/file/96894468eb44631a32d7ebd56f9892c7-Paper-Conference.pdf

[28] Bowen Pang, Kai Li, and Feifan Wang. 2025. Optimizing LLM Inference Throughput via Memory-aware and SLA-constrained Dynamic Batching. arXiv preprint arXiv:2503.05248 (2025).

[29] Ivy Bo Peng, Roberto Gioiosa, Gokcen Kestor, Pietro Cicotti, Erwin Laure, and Stefano Markidis. 2017. Exploring the Performance Benefit of Hybrid Memory System on HPC Environments. In 2017 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW). IEEE. doi:10.1109/ipdpsw.2017.115

[30] Ethan Perez, Douwe Kiela, and Kyunghyun Cho. 2021. True Few-Shot Learning with Language Models. In Advances in Neural Information Processing Systems. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2021/file/5c04925674920eb58467fb52ce4ef728-Paper.pdf

[31] Ramya Prabhu, Ajay Nayak, Jayashree Mohan, Ramachandran Ramjee, and Ashish Panwar. 2025. vAttention: Dynamic Memory Management for Serving LLMs without PagedAttention. In Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1. Association for Computing Machinery. doi:10.1145/3669940.3707256

[32] Qidong Su, Wei Zhao, Xin Li, Muralidhar Andoorveedu, Chenhao Jiang, Zhanda Zhu, Kevin Song, Christina Giannoula, and Gennady Pekhimenko. 2025. Seesaw: High-throughput LLM Inference via Model Re-sharding. arXiv:2503.06433 [cs.DC] https://arxiv.org/abs/2503.06433

[33] The Big Prompt Library Contributors. 2024. The Big Prompt Library: A Collection of Prompts, System Prompts and LLM Instructions. https://github.com/0xeb/TheBigPromptLibrary. GitHub repository.

[34] vLLM Project Contributors. 2024. Automatic Prefix Caching. https://docs.vllm.ai/en/stable/design/prefix_caching/. vLLM Documentation.

Accessed: 2026-03-10.

[35] Christopher J. C. H. Watkins and Peter Dayan. 1992. Q-Learning. Machine Learning (1992). doi:10.1007/BF00992698

[36] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, Ed H. Chi, Tatsunori Hashimoto, Oriol Vinyals, Percy Liang, Jeff Dean, and William Fedus. 2022. Emergent Abilities of Large Language Models. Transactions on Machine Learning Research (2022). https://openreview.net/forum?id=yzkSU5zdwD Survey Certification.

[37] Bingyang Wu, Shengyu Liu, Yinmin Zhong, Peng Sun, Xuanzhe Liu, and Xin Jin. 2024. LoongServe: Efficiently Serving Long-Context Large Language Models with Elastic Sequence Parallelism. In Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles. Association for Computing Machinery. doi:10.1145/3694715.3695948

[38] Jinwei Yao, Kaiqi Chen, Kexun Zhang, Jiaxuan You, Binhang Yuan, Zeke Wang, and Tao Lin. 2024. DeFT: Decoding with Flash Tree-attention for Efficient Tree-structured LLM Inference. In International Conference on Learning Representations. https://api.semanticscholar.org/CorpusID:268819748

[39] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. 2025. CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion. In Proceedings of the Twentieth European Conference on Computer Systems. Association for Computing Machinery. doi:10.1145/3689031.3696098

[40] Lu Ye, Ze Tao, Yong Huang, and Yang Li. 2024. ChunkAttention: Efficient Self-Attention with Prefix-Aware KV Cache and Two-Phase Partition. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). Association for Computational Linguistics. doi:10.18653/v1/2024.acl-long.623

[41] Jinjun Yi, Zhixin Zhao, Yitao Hu, Ke Yan, Weiwei Sun, Hao Wang, Laiping Zhao, Yuhao Zhang, Wenxin Li, and Keqiu Li. 2026. PAT: Accelerating LLM Decoding via Prefix-Aware Attention with Resource Efficient Multi-Tile Kernel. In Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2. Association for Computing Machinery. doi:10.1145/3779212.3790200

[42] Chen Zhang, Kuntai Du, Shu Liu, Woosuk Kwon, Xiangxi Mo, Yufeng Wang, Xiaoxuan Liu, Kaichao You, Zhuohan Li, Mingsheng Long, Jidong Zhai, Joseph Gonzalez, and Ion Stoica. 2025. Jenga: Effective Memory Management for Serving LLM with Heterogeneity. In Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles. Association for Computing Machinery. doi:10.1145/3731569.3764823

[43] Yilong Zhao, Shuo Yang, Kan Zhu, Lianmin Zheng, Baris Kasikci, Yifan Qiao, Yang Zhou, Jiarong Xing, and Ion Stoica. 2026. BlendServe: Optimizing Offline Inference with Resource-Aware Batching. In Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2. Association for Computing Machinery. doi:10.1145/3779212.3790133

[44] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: efficient execution of structured language model programs. In Proceedings of the 38th International Conference on Neural Information Processing Systems. Curran Associates Inc.

[45] Wanyi Zheng, Minxian Xu, Shengye Song, and Kejiang Ye. 2025. BucketServe: Bucket-Based Dynamic Batching for Smart and Efficient LLM Inference Serving. arXiv:2507.17120 [cs.DC] https://arxiv.org/abs/2507.17120

[46] Zhen Zheng, Xin Ji, Taosong Fang, Fanghao Zhou, Chuanjie Liu, and Gang Peng. 2025. BatchLLM: Optimizing Large Batched LLM Inference with Global Prefix Sharing and Throughput-oriented Token Batching. arXiv:2412.03594 [cs.CL] https://arxiv.org/abs/2412.03594

[47] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: disaggregating prefill and decoding for goodput-optimized large language model serving. In Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation. USENIX Association.

A Experimental Setup of §3.1

A.1 Two Large Prefixes - Heterogeneity

Figure 23 shows the visualization of the experiment corresponding to takeaway 1 of §3. We have a total of 500 requests, each using one of the prefixes A or B. These prefixes are 10K tokens long, and each query has a unique 20 token-long suffix. $f \in [0, 1]$ denotes the fraction of requests using prefix A. When f is either 0 or 1, the batch is homogeneous; otherwise, two prefix groups coexist.

A.2 Fractional Sharing

Figure 24 shows the visualization of the experiment corresponding to takeaway 2 of §3. We have a total of 100 requests, each with a length of 2K tokens. Each request shares a common prefix of length $2K \times f$ tokens, where $f \in [0, 1]$, with the remaining $2K \times (1 - f)$ unique.

A.3 Radix Tree Sharing Patterns

Figure 25 shows the visualization corresponding to takeaway 4 in §3. We consider four different radix tree sharing patterns. The sequence length in each case is fixed at $4L$. (i) In Figure 25(a), the entire $4L$ prefix is shared across all requests. (ii) In Figure 25(b), the tree has two levels. The first level, of length L, is shared by all requests, while the second level consists of two branches, each of length $3L$. (iii) In Figure 25(c), the tree has three levels with lengths L, L, and $2L$, and with 1, 2, and 4 branches at each level, respectively. (iv) In Figure 25(d), the tree has four levels, each of length L, with 1, 2, 4, and 8 branches at successive levels.

B Additional Results from §3.1

Below, we present a set of experiments to strengthen our observations from §3.1 regarding prefix homogeneity.

B.1 Effect of Total KV Cache Size

Figure 26 illustrates vLLM's FCFS throughput alongside the total KV cache memory footprint for prefix tokens in the Qwen 0.5B model, evaluated across varying prefix lengths and numbers of prefix groups. As expected, throughput decreases as the prefix sequence length increases. A more pronounced drop is observed when the number of prefix groups

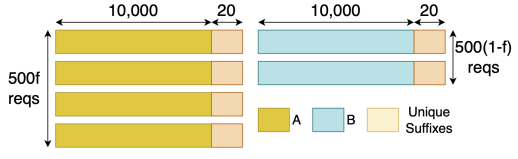


Figure 23. Two Large Prefixes

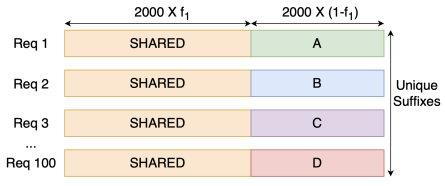
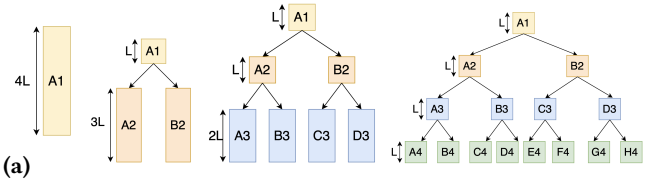
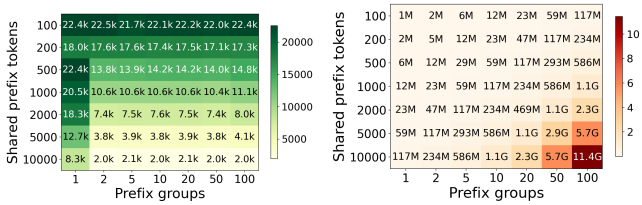


Figure 24. Fractional Sharing



(a) 1-level **(b)** 2 levels **(c)** 3 levels **(d)** 4 levels

Figure 25. Radix Tree Sharing Levels

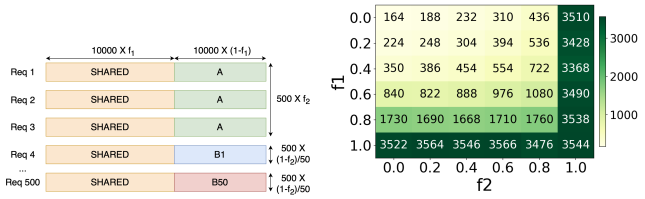


(a) Throughput **(b)** KV Cache Memory

Figure 26. Throughput and KV Cache Size

increases from 1 to 2, which can be attributed to a loss of locality. Beyond this point, however, throughput remains relatively stable. This suggests that increasing the number of prefix groups further does not significantly degrade temporal or spatial locality, indicating that most of the locality loss occurs in the transition from a single group to multiple groups. Moreover, this drop from 1 to 2 prefix groups is observable only for larger shared prefix lengths where temporal and spatial locality benefits kick in.

Another question to ask if the high throughput values are correlated with the entire KV cache working set fitting into one of the cache levels. L1 cache size is typically in KBs, which is too small even for tens of tokens. Therefore, we focus on the L2 cache, which is 96 MB in size for the GPU on which these experiments were run. In the top-left region of the heatmap, the entire KV cache fits within this size. However, no clear throughput degradation is observed once this threshold is exceeded. This indicates that effective spatial and temporal locality depend more on whether the subset of KV blocks actively accessed by batched requests at a given time (i.e. the dynamic working set) fits in the L2 cache. It is intuitive that this dynamic working set is smaller in size for prefix-homogeneous batches which explains the higher throughput gains achieved for such batches. Moreover, even for such a small model, the model weights are already large enough to prevent the entire KV cache to reside in L2. Therefore, performance gains due to prefix homogeneity primarily stem from a smaller dynamic working set of the KV cache.



(a) Visualization **(b)** Throughput

Figure 27. Key Observations on Prefix Homogeneity

B.2 Key Observations on Prefix Homogeneity

We consider an additional experiment where each request has a sequence length of $10K$ tokens. As illustrated in Figure 27(a), a fraction $f_1 \in [0, 1]$ of this sequence is shared as a common prefix across all 500 requests in the batch. The remaining $10K(1 - f_1)$ tokens constitute the suffix of each request. Within the batch, a fraction $f_2 \in [0, 1]$ of requests share an identical suffix, denoted as A. The remaining fraction $(1-f_2)$ has suffixes drawn from one of 50 distinct groups, $B_1, \ldots, B_{50}$. Figure 27(b) presents the throughput as we vary f_1 and f_2. Consistent with the observations in §3.1, increasing f_1 improves throughput due to enhanced temporal and spatial locality in KV cache accesses. Additionally, we observe a sharp increase in throughput when f_2 changes from 0.8 to 1. This occurs because at $f_2 = 1$, the entire batch becomes homogeneous, resulting in fully aligned KV cache traversal. Throughput is significantly lower at smaller values of f_1, primarily because limited GPU memory leads to frequent cache evictions. This is further supported by the increase in throughput with rising f_2 at low f_1, as more requests share the suffix A, reducing eviction frequency. However, once the working set fits within GPU memory (around $f_1 = 0.8$), throughput remains nearly constant for all $f_2 < 1$, indicating that residual heterogeneity has minimal impact. Overall, this experiment consolidates the key insights discussed in §3.1.

C Operations of Chunked Hash Tree

This section provides pseudo-codes and details of various algorithms used in the Chunked Hash Tree (§4.2).

C.1 Variables and Notations

Table 2 provides a list of all the variables and notations used in the following pseudo-codes.

C.2 Prefix Hash Computation

The goal is to efficiently compute prefix-aligned hashes at fixed chunk boundaries. Algorithm 1 computes the hash vector for a token sequence using an incremental hashing scheme (Figure 28). Instead of recomputing hashes for each prefix independently, the algorithm maintains a streaming hash state that is updated once per token. The algorithm processes each token exactly once and performs constant-time updates to the hash state. As a result, it runs in $O(T)$ time and produces $C = \lceil T/K \rceil$ hash values, yielding $O(C)$

Variable	Explanation
T	Sequence length in tokens
C	Number of chunks in a sequence
K	Chunk size
H	Hash vector
r	Request
W	Number of waiting requests
W_h	Average number of waiting requests sharing a hash
(l^*, h^*)	Shared prefix tip
h_l^r	Hash at level l of request r
`working_set`	Working set
`waiting`	Reverse index from hashes to waiting requests
`waiting_heap`	Min-heap
`request_hashes`	Map from request to its hash vector
`miss`	Map from request to its missing count
`active`	Set of requests present in the current batch
`waiting_requests`	Set of waiting requests
`ref`	Map from hash to number of active requests sharing it

Table 2. Variables and Notations Used

output space. While Algorithm 1 processes tokens sequentially, the prefix-hash vector can equivalently be computed in parallel across chunks. The key property we want is that all requests sharing the first $c \cdot K$ tokens produce an identical h_c. This is achieved by partitioning the token sequence into chunks $S_c = (t_{(c-1)K+1}, \ldots, t_{cK})$, hashing each chunk independently in parallel, and then chaining the results via $h_c = \textsc{Hash}(h_{c-1} \,\|\, \textsc{Hash}(S_c))$, with h_0 set to a fixed initialization constant (Algorithm 2). With parallel workers, the dominant hashing phase runs in $O(K)$ per worker rather than $O(T)$ sequentially, reducing overall wall-clock time to $O(K + C)$; the subsequent chaining pass costs only $O(C)$ and is negligible in practice. This parallelization is most beneficial when K is large, as each chunk contains more tokens and therefore represents a heavier independent unit of work, allowing parallel workers to amortize scheduling overhead across a larger payload and yielding greater absolute speedup over the sequential baseline. Both algorithms produce a hash vector satisfying the prefix-consistency property—all requests sharing the first $c \cdot K$ tokens yield the same h_c, yet the two vectors differ in their actual hash values, as Algorithm 2 is a distinct formulation rather than a parallelized execution of Algorithm 1.

Algorithm 1 ComputeHashes(S, K)

Require: Token sequence $(t_1, \ldots, t_T)$; chunk size K
Ensure: Cumulative prefix-hash vector $H = (h_1, \ldots, h_C)$

```
1: H ← []; C ← ⌈T/K⌉
2: Initialise incremental xxHash-64 state s
3: for i ← 1 to T do
4:     Feed t_i into s
5:     if i mod K = 0 or i = T then
6:         H.Append(Digest(s))
7:     end if
8: end for
9: return H
```

Algorithm 2 ComputeHashesParallel(S, K)

Require: Token sequence $(t_1, \ldots, t_T)$; chunk size K
Ensure: Cumulative prefix-hash vector $H = (h_1, \ldots, h_C)$

```
1: C ← ⌈T/K⌉
2: h_0 ← InitConst
3: parallel for c ← 1 to C do   ▷ Independent per-chunk hashes
4:     S_c ← (t_{(c−1)K+1}, ..., t_{min(cK,T)})
5:     ℓ_c ← Hash(S_c)          ▷ Each chunk hashed independently
6: end parallel for
7: for c ← 1 to C do            ▷ Sequential prefix chaining
8:     h_c ← Hash(h_{c−1} ‖ ℓ_c)
9: end for
10: return H = (h_1, ..., h_C)
```

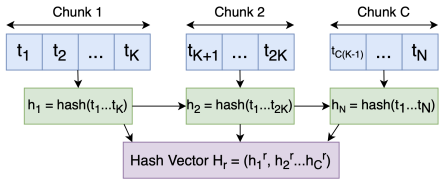


Figure 28. Hash Computation in Chunked Hash Tree

C.3 Insertion

Algorithm 3 provides the pseudo-code for the Insert procedure. Given a request with a token sequence, we compute its cumulative prefix-hash vector using ComputeHashes and store it in a dictionary that maps each request to its hashes for easier reuse later (lines 1-2). For each level, we retrieve the corresponding hash and check whether the tuple is present in the working set; if not, the missing count is incremented (lines 6-7). Simultaneously, the request is inserted into the reverse index (line 9), which maps each hash to the set of waiting requests containing it and supports efficient lookup of prefix-sharing candidates. After processing all levels, we store the miss count in a map and insert the tuple of miss count and the request into the min-heap, which is keyed by missing count, enabling efficient retrieval of the most compatible request (lines 11-12). The design avoids any updates to the working set or reference counts during insertion, ensuring low overhead.

C.4 Finding the Best Request

The FindBest procedure selects the most compatible waiting request by querying the min-heap while skipping stale entries arising from lazy updates until a valid minimum is found (lines 6-8). Once a candidate is identified, it is reinserted into the heap for future activations in case the RL policy decides to add this request to the batch (line 9), and three quantities are computed without modifying the system state: the current tip depth, the prospective depth, and the number of peers sharing the resulting prefix. The prospective

Algorithm 3 INSERT(r, S)

Require: r; token sequence S; *working_set*; *waiting*; *waiting_heap*
Ensure: r registered and ready for scheduling

```
1:  H_r ← ComputeHashes(S, K)
2:  request_hashes[r] ← H_r
3:  m_r ← 0
4:  for ℓ ← 1 to |H_r| do
5:      h ← h_ℓ^r
6:      if (ℓ, h) ∉ working_set then
7:          m_r ← m_r + 1
8:      end if
9:      waiting[h] ← waiting[h] ∪ {r}
10: end for
11: miss[r] ← m_r
12: waiting_heap.Push(m_r, r)
```

Algorithm 4 FINDBEST()

Require: *waiting_heap*; *miss*; *active*; tip (ℓ^*, h^*); *working_set*; *request_hashes*
Ensure: $(r^*, \ell^*_{before}, \ell^*_{after}, peers)$ or $\perp$

```
1:  if waiting_requests = ∅ then return ⊥
2:  end if
3:  if cache_valid then return cached_result
4:  end if
5:  while waiting_heap ≠ ∅ do
6:      (m, r) ← waiting_heap.Pop()
7:      if r ∈ active or r ∉ miss or miss[r] ≠ m then
        continue
8:      end if
9:      r* ← r;  waiting_heap.Push(m, r*)   ▷ restore for
        future calls
10:     ℓ*_before ← ℓ*
11:     ℓ*_after ← ℓ where ℓ is the largest ℓ ≤ min(ℓ*, |H_r*|)
        s.t. (ℓ, h_ℓ^r*) ∈ working_set
12:     peers ← |{w ∈ waiting_requests : h^w_ℓ*after = h^r*_ℓ*after }|
13:     Cache and return (r*, ℓ*_before, ℓ*_after, peers)
14: end while
15: return ⊥
```

depth is obtained via a backward scan over the cumulative hash vector, selecting the deepest level such that the corresponding level-hash tuple exists in the working set, which corresponds to the least common ancestor with the active set under the current state. The peer count is computed by counting waiting requests whose hash at that level matches that of the candidate request. The result is cached and reused until either the working set or the waiting queue changes, avoiding redundant recomputation across repeated calls. If no valid request is found, the procedure returns $\perp$.

Algorithm 5 ADDTOBATCH(r)

Require: H_r; ref; *working_set*; *waiting*; *miss*; *waiting_heap*; tip (ℓ^*, h^*); *active*
Ensure: Updated ref, *working_set*, *miss*, (ℓ^*, h^*)

```
1:  active ← active ∪ {r}; waiting_requests ← waiting_requests \ {r}
2:  if |active| = 1 then
3:      (ℓ*, h*) ← (|H_r|, h^r_|H_r|);  return
4:  end if
5:  ℓ_tip ← min(ℓ*, |H_r|);  (ℓ*, h*) ← (0, ⊥)
6:  for ℓ ← |H_r| downto 1 do
7:      if ref[h_ℓ^r] = 0 then
8:          working_set ← working_set ∪ {(ℓ, h_ℓ^r)}
9:          for w ∈ waiting[h_ℓ^r], w ∉ active do
10:             miss[w] −= 1;
11:             waiting_heap.Push(miss[w], w)
12:         end for
13:     end if
14:     ref[h_ℓ^r] += 1
15:     if ℓ* = 0 and ℓ ≤ ℓ_tip and (ℓ, h_ℓ^r) ∈ working_set
        then
16:         (ℓ*, h*) ← (ℓ, h_ℓ^r)   ▷ Least common ancestor
        found; continue loop for remaining working set updates
17:     end if
18: end for
```

C.5 Adding a Request to the Active Batch

Given a request with its prefix-hash vector, the ADDTOBATCH procedure promotes it to the active set (line 1) and updates the working set, reference counts, and other metadata in a single backward pass over the prefix-hash vector. For each level, if the reference count of the corresponding hash is zero (no other active request shares it), the level-hash tuple is inserted into the working set, and all corresponding waiting requests have their miss counts decremented and are reinserted into the waiting heap (lines 7-12). The reference counts are incremented at all levels (line 14). In parallel, the shared prefix tip is recomputed by tracking the deepest level at which the level-hash tuple remains in the working set; the first such match during the backward traversal determines the new tip (lines 15-17), while the loop continues to ensure that all state updates are applied. If this is the first active request, the tip is initialized directly to the end of its prefix-hash vector.

C.6 Removing a Request from the Active Batch

Given a completed request, the FINISH procedure removes it from the active set (line 1) and updates the working set, reference counts, and other metadata. A forward pass over the prefix-hash vector decrements the reference count at each level (line 3); if it drops to zero, the level-hash tuple is removed from the working set, and all the corresponding

Algorithm 6 FINISH(r)

Require: H_r; ref; *working_set*; *waiting*; *miss*; *waiting_heap*; tip (ℓ^*, h^*); *active*
Ensure: Updated ref, *working_set*, *miss*, (ℓ^*, h^*)

1: $active \leftarrow active \setminus \{r\}$
2: **for** $\ell \leftarrow 1$ **to** $|H_r|$ **do**
3: ref$[h_\ell^r]$ −= 1
4: **if** ref$[h_\ell^r] = 0$ **then**
5: $working_set \leftarrow working_set \setminus \{(\ell, h_\ell^r)\}$
6: **for** $w \in waiting[h_\ell^r],\ w \notin active$ **do**
7: $miss[w]$ += 1;
8: $waiting_heap$.PUSH($miss[w], w$)
9: **end for**
10: **end if**
11: **end for**
12: **if** $active = \emptyset$ **then**
13: $(\ell^*, h^*) \leftarrow (0, \perp)$; **return**
14: **end if**
15: **if** $|active| = 1$ **then**
16: let q be the sole element of *active*
17: $(\ell^*, h^*) \leftarrow (|H_q|,\ h^q_{|H_q|})$; **return**
18: **end if**
19: **while** $\ell^* + 1 \le \min_{q \in active} |H_q|$ **do** ▷ try to extend tip forward
20: $\ell \leftarrow \ell^* + 1$
21: **if** ref$[h_\ell^{q_0}] = |active|$ for arbitrary $q_0 \in active$ **then** ▷ all active requests agree at level ℓ
22: $(\ell^*, h^*) \leftarrow (\ell,\ h_\ell^{q_0})$
23: **else**
24: **break**
25: **end if**
26: **end while**
27: REMOVE(r) ▷ cleans reverse index and request hashes

waiting requests have their miss counts incremented and are reinserted into the min-heap (lines 4-10). After updating the working set, the shared prefix tip is recomputed by attempting to extend it forward: starting from the previous tip level, we check successive levels up to the minimum length among active requests and extend the tip whenever the reference count at that level equals the number of active requests, indicating agreement across all of them; the scan terminates at the first disagreement (lines 19-26). If the active set becomes empty, the tip is reset to $(0, \perp)$ (lines 12-14), and if only one request remains, the tip is set to its full length (lines 15-18). Finally, REMOVE is invoked to clean auxiliary structures such as the reverse index and the stored hash vector (line 27).

C.7 Batch Formation

BUILDBATCH describes the entire pipeline, where we incrementally construct a batch by repeatedly querying FINDBEST

Algorithm 7 BUILDBATCH()

Require: Waiting queue Q; policy π (heuristic, bandit, or Q-learning); discount γ; learning rate α
Ensure: Batch B ready for one decode iteration

1: $B \leftarrow \emptyset$
2: **while** $Q \neq \emptyset$ **do**
3: $(r^*, c_{\text{before}}, c_{\text{after}}, w) \leftarrow$ FINDBEST()
4: **if** $r^* = \perp$ **then break**
5: **end if**
6: $s \leftarrow (|B|,\ \Delta = c_{\text{before}} - c_{\text{after}},\ w)$
7: $a \leftarrow \pi(s)$ ▷ heuristic rule / UCB look-up / ε-greedy Q-table
8: **if** $a =$ STOP **then break**
9: **end if**
10: ADDTOBATCH(r^*); $B \leftarrow B \cup \{r^*\}$
11: **end while**
12: $\text{throughput}_{\text{decode}} \leftarrow$ EXECUTEBATCH(B)
13: **Update** π with observed reward $\text{throughput}_{\text{decode}}$ ▷ no-op for heuristic
14: **return** B

to obtain the most compatible candidate, along with summary statistics, which are combined with the current batch size to form the RL state (lines 3-6). A policy (heuristic, bandit, or Q-learning) maps this state to an action, either ADD or STOP (line 7); if ADD is selected, the request is activated via ADDTOBATCH and appended to the batch; otherwise, the loop terminates (lines 8-10). The procedure continues until the waiting queue is exhausted or the policy decides to stop, after which the constructed batch is executed for one decode iteration to obtain the observed throughput (line 12). This throughput serves as the reward signal used to update the policy (line 13), with bandit methods updating their statistics and Q-learning applying a Bellman update with parameters α and γ, while heuristic policies incur no update.

C.8 Complexity Analysis

Table 3 summarizes the time complexity of each operation. Tip maintenance in ADDTOBATCH and FINISH is subsumed within their $O(C \cdot W_h \cdot \log W)$ passes and adds no asymptotic cost. In practice, the dominant cost is miss-count propagation, which updates all affected waiting requests per working-set change. Result caching avoids redundant FINDBEST traversals between mutations, keeping scheduling overhead low. We present the time taken by individual functions for a particular experimental setup in §D.1.

C.9 Find Best Request - Alternative Heuristic

An intuitive alternative to using the missing count (which is based on the difference with the working set) is to select a waiting request that maximizes the depth of the current shared prefix tip. However, this approach has a subtle flaw:

Operation	Complexity
Insert	$O(T + C \log W)$
AddToBatch	$O(C \cdot W_h \cdot \log W)$
Finish	$O(C \cdot W_h \cdot \log W)$
FindBest	$O(\log W)$ amortized; $O(1)$ cached
Remove	$O(C)$

Table 3. Time complexity of CHT scheduler operations

the tip is inherently bottlenecked by the most divergent active request, limiting future extension. Consider a scenario where Level 1 is shared by all requests, but Level 2 splits into three branches: one for R_1, one shared by R_2 and R_3, and one for R_4 (Figure 29). If the active batch contains R_1 and R_2, the shared prefix tip is restricted to Level 1. Selecting based only on this tip makes R_3 and R_4 appear equally viable. If the scheduler admits R_4, the tip remains at Level 1; when R_1 finishes, the remaining requests (R_2 and R_4) lie on divergent branches, leaving the tip still stuck. Our Chunked Hash Tree avoids this by evaluating candidates against the full working set using the missing count m_r, rather than anchoring to the bottlenecked tip. The working set tracks all prefix chunks present in any active request. In the example, since R_2 is active, its Level 2 chunk is already in the working set. Thus, R_3 requires fewer missing chunks than R_4, i.e., $m_{R_3} < m_{R_4}$. Selecting R_3 preserves future overlap: although the immediate tip remains bottlenecked, once R_1 finishes and is removed via Finish, the remaining batch $\{R_2, R_3\}$ shares a deeper prefix, and the tip extends to Level 2.

So, we have two ways to find the best waiting request to be added to the batch. One is based on the difference from the working set, and the other is based on the shared prefix tip. Next, we try to formalize the metrics on which both of these ways are based. Let W be the hashes in the current working set, which can be defined as: $\cup_{r \in A} H^r$, where A is the active set of requests and H^r is the set of hashes for request r. CHT selects the request that has the minimum miss-count, which can be defined as $m_W(r) = |H^r \setminus W|$. The shared prefix tip is the deepest level shared by all the requests of the active batch, which can be defined as $\max\{\ell : \forall r \in A, (\ell, h^r_\ell) \in W\}$. The alternative heuristic selects the request that leads to the maximum depth of the shared prefix tip, which is equivalent to minimizing the number of hashes missing from the working set at or above the current tip for that request. Let $m_S(r) = |S \setminus H^r|$, where S is the set of all hashes at or above the current tip level in the working set, i.e., $S = \{h : (\ell, h) \in W, \ell \leq \ell^*\}$, where ℓ^* is the current tip level. That is, $m_S(r)$ denotes the number of hash chunks present in S, but not present in the request r. Therefore, we explore the alternative heuristic that minimizes $m_S(r)$ instead of $m_W(r)$.

Let us assume that all the requests have a length of C chunks, and the best requests emitted from the two metrics are r_W and r_S, respectively. Let us also assume that the shared prefix tip is at level ℓ^*. We aim to show that $m_S(r_W) = m_S(r_S)$, that is, the working set metric causes equal loss in the shared prefix tip, implying that r_W does not lead to less depth in the shared prefix tip compared to r_S. Let us assume, to the contrary, that $m_S(r_S) < m_S(r_W)$. Now, we compare $m_W(r_W)$ and $m_W(r_S)$ and aim to show that $m_W(r_W) > m_W(r_S)$, which contradicts the fact that r_w minimizes the metric m_W.

Since S represents the shared prefix of all active requests up to depth ℓ^*, the working set W contains exactly one unique hash per level for all levels $\ell \leq \ell^*$, which constitutes the set S. This property dictates that if any request x misses a hash in S at some level $k < \ell^*$, its prefix diverges from the *entire* active batch at level k. Consequently, due to the cumulative nature of the chunked hashes, x cannot share any hashes with W at any level $\geq k$. Specifically, if a request does not fully match the shared tip (i.e., $m_S(x) > 0$), it cannot overlap with any deeper divergent branches in W. For such a request, its overlap with the working set is exactly its overlap with the shared tip: $|H^x \cap W| = |H^x \cap S| = \ell^* - m_S(x)$. Using our contrary assumption $m_S(r_S) < m_S(r_W)$, we evaluate the cost of both requests under the working set metric, $m_W(x) = C - |H^x \cap W|$. We consider two cases for r_S:

Case 1: r_S fully matches the shared prefix tip ($m_S(r_S) = 0$). Since r_S matches S entirely, $|H^{r_S} \cap S| = \ell^*$. It may also overlap with deeper branches in W, meaning $|H^{r_S} \cap W| \geq \ell^*$. Therefore, its cost is $m_W(r_S) \leq C - \ell^*$. Because $m_S(r_W) > m_S(r_S) = 0$, r_W diverges before the tip. As established, this means $|H^r_W \cap W| = \ell^* - m_S(r_W) < \ell^*$. Thus, $m_W(r_W) = C - |H^r_W \cap W| > C - \ell^*$. Combining these inequalities yields: $m_W(r_S) < m_W(r_W)$ (Figure 30(a)).

Case 2: r_S diverges before the shared prefix tip ($m_S(r_S) > 0$). Since $m_S(r_W) > m_S(r_S) > 0$, both requests diverge before ℓ^*. Their overlap with the working set is strictly limited to their overlap with S. Thus, $|H^{r_S} \cap W| = \ell^* - m_S(r_S)$ and $|H^r_W \cap W| = \ell^* - m_S(r_W)$. Because $m_S(r_S) < m_S(r_W)$, it logically follows that $|H^{r_S} \cap W| > |H^r_W \cap W|$. Consequently: $m_W(r_S) < m_W(r_W)$ (Figure 30(b)).

In both cases, assuming $m_S(r_S) < m_S(r_W)$ unavoidably leads to $m_W(r_S) < m_W(r_W)$. However, this directly contradicts our initial premise that r_W is the optimal request chosen by the Chunked Hash Tree metric (which guaranties $m_W(r_W) \leq m_W(q)$ for all waiting requests q). Therefore, our assumption must be false, proving that $m_S(r_W) \leq m_S(r_S)$. Because r_S is defined as the absolute minimum for m_S, it must strictly hold that $m_S(r_W) = m_S(r_S)$. This completes the proof that optimizing for the working set metric is mathematically guaranteed to be at least as optimal as the shared prefix tip metric for preserving the immediate tip depth while simultaneously exposing deeper amortization opportunities in $W \setminus S$.

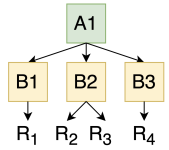
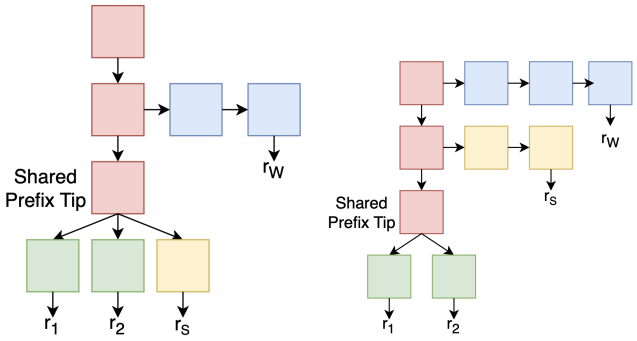


Figure 29. FindBest - Alternative Heuristic Example



(a) Case 1 **(b)** Case 2

Figure 30. FindBest cases: r_1 and r_2 are both present in the active batch, and hence the working set is the union of all of their chunks. The shared prefix tip is at the third level, and the set S of all shared prefix chunks are marked in red. In the first case, r_S fully matches the shared prefix tip, whereas r_W hangs from the middle. For this case, $m_W(r_W) = 2, m_W(r_S) = 1, m_S(r_W) = 1, m_S(r_S) = 0$. In the second case, both the requests r_S and r_W do not fully match the shared prefix tip, however r_W hangs off from an earlier chunk. For this case, $m_W(r_W) = 3, m_W(r_S) = 2, m_S(r_W) = 2, m_S(r_S) = 1$. Therefore, for both the cases, m_W and m_S exhibit similar behaviour.

D Additional Results of Feather

Below, we present additional evaluation results of Feather that could not be included in the main body due to space constraints.

D.1 Time Taken by Individual Functions of CHT

Table 4 profiles CHT operations over an inference workload of $5K$ requests, each $5K$ tokens long. Insert, AddToBatch, and Finish are each invoked $5K$ times, once per request. FindBest, however, is called 26,405 times because it is queried repeatedly during batch construction. The RL policy may reject candidates and trigger fresh lookups, and the scheduler may inspect the queue multiple times before committing to a Stop. Insert dominates total time because hash computation scans every token once, making it proportional to prompt length. Finish is more expensive than AddToBatch, despite their symmetric roles. This gap arises from Finish's additional forward tip-extension scan. After updating the working set, it traverses forward, level by level, to check if the shared prefix can be extended. In contrast, AddToBatch performs an early-terminating backward LCA scan, avoiding this overhead. FindBest is the cheapest per-call operation due to result caching and lazy heap traversal.

Function	Number of Calls	Average Time (ns)	Total Time (ns)
Insert	5,000	153,714	768,570,573
AddToBatch	5,000	11,143	55,715,469
Finish	5,000	38,952	194,760,111
FindBest	26,405	6,641	175,372,986

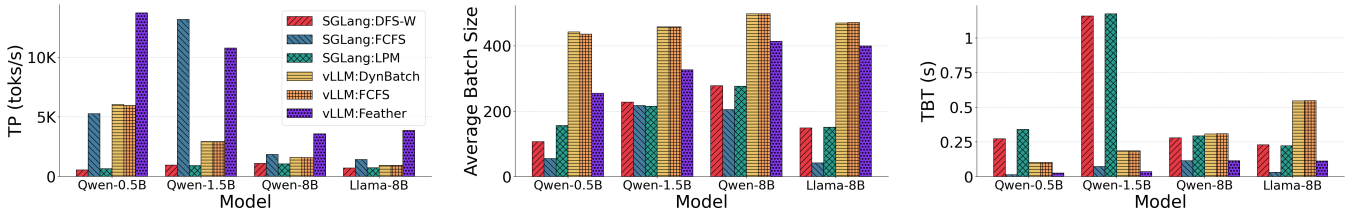
Table 4. Time taken by individual functions of CHT

D.2 Performance of Feather across Models

Figures 31(a), 31(b), 31(c) present throughput, average running batch size, and time between tokens (TBT) across models of varying sizes. The overall trend is consistent with expectations: smaller models achieve higher throughput owing to lower per-token compute costs. Feather outperforms all baselines across all models. The sole exception is Qwen-1.5B, where SGLang FCFS achieves marginally higher throughput; however, Feather compensates on the latency axis, exhibiting substantially lower TBT for the same model. A key structural observation is that Feather's relative throughput advantage over the baselines grows with model size. As model size increases, a larger fraction of the total execution time is spent on attention computation, which amplifies the benefits of temporal and spatial KV-cache locality that Feather exploits. This effect is visible in Figure 31(a): the gap between Feather and the next-best baseline widens from Qwen-0.5B to the 8B-scale models. The performance of SGLang's LPM and DFS-W schedulers degrades noticeably for smaller models. Both policies rely on CPU-side tree traversal, whose overhead scales with sequence length rather than model size; as the model shrinks, GPU execution time decreases while scheduler overhead remains roughly constant, causing the CPU to become an increasing bottleneck. Feather avoids this through its Chunked Hash Tree, which reduces per-step scheduler complexity and keeps CPU overhead negligible relative to GPU execution time across all model scales. Finally, Figure 31(c) highlights that vLLM's FCFS and DB baselines incur substantially higher TBT at larger model sizes, whereas Feather keeps TBT low and stable.

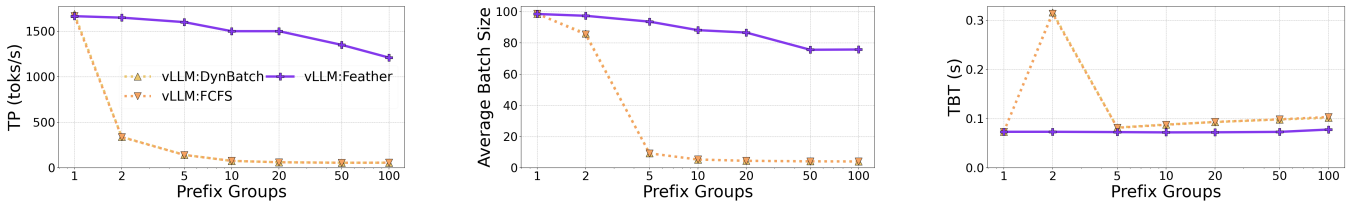
D.3 Results across LongChat Model

Figure 32 presents throughput, average batch size, and time between tokens (TBT) as the number of prefix groups varies for the LongChat 13B model. Overall, vLLM's FCFS and Dynamic Batching behave similarly across most configurations. For FCFS, there is a sharp drop in throughput when moving from 1 to 2 prefix groups, accompanied by an increase in TBT. This is largely due to the loss of locality benefits. The further decline from 2 to 5 prefix groups is driven by KV cache evictions, which is also reflected in the steep reduction in average batch size. Beyond this point, TBT stabilizes at a lower value, primarily because the batches have become much smaller. In contrast, Feather maintains stable and consistent throughput across all settings. At 100 prefix groups, it achieves up to 22× higher throughput than FCFS. By forming prefix-homogeneous batches, Feather not only improves



(a) Throughput **(b)** Average running batch size **(c)** Time between tokens (TBT)

Figure 31. Different Models



(a) Throughput **(b)** Average Batch Size **(c)** Time between tokens (TBT)

Figure 32. Varying number of prefix groups for LongChat 13B

scheduling efficiency but also significantly reduces KV cache evictions through more effective immediate cache reuse.

D.4 Complexity Analysis of LPM and DFS-W

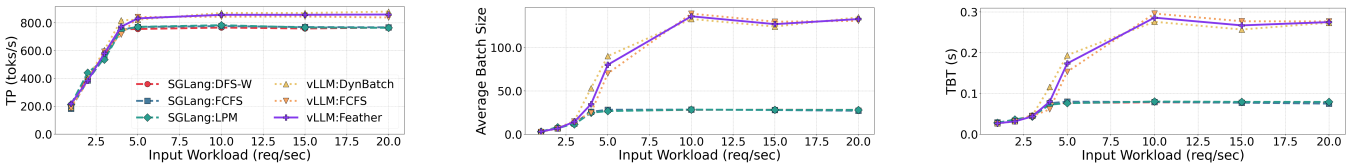
We provide a detailed complexity analysis of the two SGLang cache-aware scheduling policies discussed in §5: Longest Prefix Match (LPM) and Depth-First Search with Weighting (DFS-W). Let W denote the number of requests in the waiting queue and T the maximum prefix length. For each request, LPM performs prefix matching on the global radix tree, costing $O(T)$ per request, and then sorts the queue by matched prefix length, adding $O(W \log W)$. The total per-round cost is therefore $O(W \cdot T + W \log W)$. The result is overhead that scales *multiplicatively* with both queue size and context length. LPM implicitly acknowledges this by falling back to FCFS when $W > 128$ (which was disabled in our experiments). DFS-W avoids per-request traversal; instead, it performs a full traversal of the radix tree in each scheduling round. Let B be the total number of radix tree nodes. DFS-W first propagates subtree request counts upward in $O(B)$, then re-traverses in a depth-first manner, sorting children by weight at every level. The total cost is $O(W+B \log B)$. The key problem is that B reflects the server's accumulated history, not the current workload. The radix tree grows monotonically and is only pruned under memory pressure, so DFS-W's overhead increases over time, regardless of the current load. A server with a large accumulated cache ($B \gg W$) will incur high scheduling costs even during quiet periods. In summary, a burst of long-context requests penalizes LPM through the $W \cdot T$ term, while a large accumulated cache penalizes DFS-W through the $B \log B$ term. Either way, scheduling overhead can reach the same order of magnitude as GPU execution time. This demonstrates that identifying shared prefixes at runtime is fundamentally expensive and naive cache-aware scheduling can become the dominant bottleneck.

D.5 Convergence of Bandit Policy

We also evaluate how quickly Feather's bandit-based policy converges to a stable and efficient batching configuration. We construct a workload with five prefix groups, each containing 5K tokens, and issue requests at a rate of 5 per second. Figure 34 illustrates how the running batch size evolves over time for Feather under its bandit-based policy. At the beginning, the system enters an exploration phase, during which Feather experiments with different configurations and reaches relatively large batch sizes (around 500). However, these batches are heterogeneous. After this phase concludes (around 100 seconds), the system shifts to homogeneous batching, which yields higher throughput. As a result, Feather converges to a significantly smaller batch size and maintains it for the remainder of the experiment.

D.6 RL Policy across Varying Workload

We generate a workload consisting of five prefix groups, each containing 5K tokens. The input request rate varies over time—from 5 to 10 to 20 requests per second—and we measure the average batch size as a function of time for Feather using its Bandit policy (Figure 35). From 0 to 200 seconds, the request rate is 5, and the batch size remains relatively low. Between 200 and 300 seconds, the rate increases to 10, leading to a noticeable rise in batch size. Finally, from 300 to 350 seconds, the request rate reaches 20, and the batch size peaks. (Fluctuations in the plot are due both to RL exploration and the workload being Poisson.) These results show that Feather dynamically adapts the batch size in response to both prefix homogeneity (i.e., the number of requests



(a) Throughput **(b)** Average Batch Size **(c)** Time between tokens (TBT)

Figure 33. Workload with No Prefix Sharing

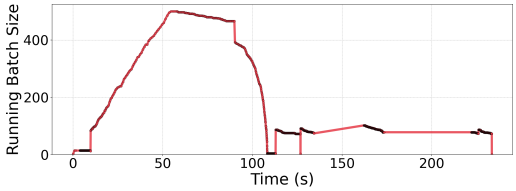


Figure 34. Convergence of Bandit Policy

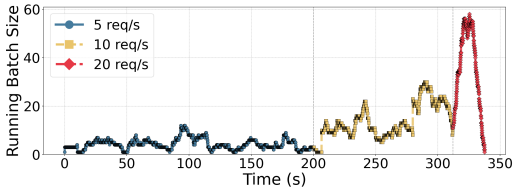


Figure 35. Performance of Bandit across Varying Workload

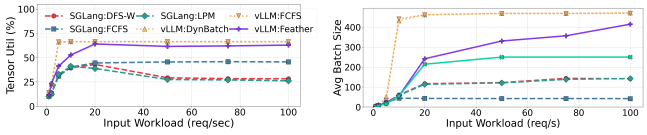
sharing the same prefix group) and the current workload intensity.

D.7 Workloads with No Prefix Sharing

We next evaluate Feather and compare it against other scheduling policies on a workload without prefix sharing; that is, no two requests share a common prefix. Figure 33 reports throughput, average batch size, and time between tokens (TBT) across a range of input rates. At higher input rates, vLLM-based policies begin to outperform SGLang policies, primarily due to their ability to form larger batches. Notably, even in the absence of prefix sharing, Feather, powered by its adaptive RL-based policy—matches the performance of vLLM's FCFS scheduler. In contrast, SGLang policies achieve lower TBT, which can be attributed to their smaller batch sizes.

D.8 Tensor Core Utilization across Policies

Figure 36 compares tensor core utilization and average batch size across different request rates and scheduling policies. Feather shows slightly lower utilization at low request rates due to forming smaller batches, but it still delivers higher throughput at these points (as seen earlier in Figure 12(a)). As the workload increases, Feather gradually builds larger batches from prefix-homogeneous requests, and its utilization rises to match vLLM's FCFS. In contrast, SGLang's FCFS maintains relatively low utilization because it operates with consistently small batch sizes. Interestingly, even though SGLang's DFS-W and LPM achieve larger batch sizes, their utilization remains low. This is mainly due to CPU-side stalls



(a) Tensor Core Utilization **(b)** Average Batch Size

Figure 36. Input Workload

between GPU executions, during which the GPU stays idle, effectively reducing overall utilization.

E Why the name Feather?

Feather is named after the structural pattern of prefix-homogeneous batches in our system: a shared prefix forms a central shaft, from which individual requests branch out as they diverge, much like the barbs of a Feather. The metaphor captures both the visual structure of our batches and the functional benefit of keeping prefix sharing requests together for better performance.